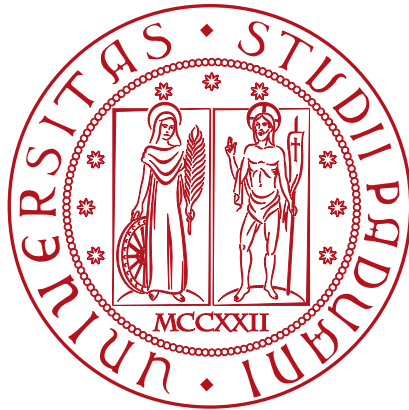


Università degli Studi di Padova

DIPARTIMENTO DI MATEMATICA "TULLIO LEVI-CIVITA"

CORSO DI LAUREA IN INFORMATICA



**Migrazione al protocollo MQTT in un
sistema di monitoraggio e tracciamento
di linee produzione basato su dispositivi
RFID**

Tesi di Laurea Triennale

Relatore

Prof. Marco Zanella

Laureando

Luca Ribon

Matricola 2075516

“Pointers are really good for pointing to things.”

— Bjarne Stroustrup

Sommario

Il presente documento descrive il lavoro svolto durante il periodo di stage, della durata di circa 320 ore, dal laureando Luca Ribon presso l'azienda KanbanBOX S.r.l.

L'obiettivo finale dello stage è stato quello di sostituire il protocollo HTTP_G, utilizzato dalla piattaforma KanbanBOX per comunicare con i lettori RFID_G, con MQTT_G, un protocollo più adatto ad essere utilizzato con dispositivi *IoT*.

In particolare è stata progettata un'architettura capace di integrare i seguenti componenti:

- **lettore RFID**, dispositivi hardware che leggono i tag RFID applicati sui prodotti;
- **AWS IoT Core**, un servizio *cloud* che integra un broker_G capace di gestire la comunicazione tra molteplici dispositivi tramite MQTT;
- **AWS SQS**, un servizio *cloud* che fornisce delle code di messaggi per la comunicazione asincrona tra servizi, particolarmente adatto a gestire la lettura dei tag tramite meccanismo *publish/subscribe* di MQTT;
- **KanbanBOX**, la piattaforma *web* che necessita di ricevere i dati letti dai lettori RFID per monitorare e supportare l'intero processo produttivo e logistico di un'azienda.

A tal fine, alla fase di progettazione è seguita la codifica delle funzionalità necessarie in KanbanBOX per l'integrazione con i servizi AWS IoT Core e SQS, che saranno di supporto per l'implementazione della comunicazione con i lettori tramite protocollo MQTT.

Ringraziamenti

Innanzitutto, vorrei sottolineare la mia stima nei confronti del Prof. Marco Zanella relatore della mia tesi, che ha sempre saputo essere disponibile e puntuale in tutte le fasi di avanzamento del mio progetto di stage e durante la stesura della relativa tesi.

Desidero ringraziare con (più o meno) affetto la mia famiglia, in particolare i miei genitori Mauro e Vania, mia sorella Sara, Tommy e GIOELE (scritto in maiuscolo così forse riesce a leggerlo) per il sostegno e la comprensione dimostrati durante tutto il percorso universitario, ma anche per l'attenzione prestata, spesso (ma non sempre), nel farmi procedere sui miei passi, assecondando le mie idee e le mie intenzioni anche quando non erano per nulla condivise.

Infine voglio (voglio?) dedicare uno spazio a tutte queste persone con gusti dubbi in fatto di amicizie che, per mia pigrizia, verranno suddivise in insiemi:

- **quelli dell'uni:** che hanno sempre trovato il modo di supportarmi, anche quando nemmeno loro avevano idea di cosa stessi facendo, senza mai ricevere abbastanza in cambio, e che, come se non bastasse, hanno riempito di ricordi questi anni di triennale (che è durata 4 anni, magia...), sia nei momenti più impegnativi che nei tanti momenti di svago...spero che per loro valga lo stesso
- **quelli dell'Islanda:** con cui è evidente che io sia stato in Islanda, il viaggio che, per me, è il più memorabile fatto fin'ora; preferisco non scrivere menzogne nella mia tesi quindi non dirò che siano mai stati di aiuto durante questo percorso ma voglio comunque dedicare loro qualche parola (senza esagerare che tanto sono sprecate) perché, dal mio punto di vista, nella propria vita è importante assumere una dose frequente di maleducazione e violenza per rompere la monotonia (apprezzata) di gentilezza e affetto che caratterizza buona parte delle persone che mi circonda (ma non me)
- **quelli del Covo (anche Asia che forse mi odia):** con cui negli anni ho avuto un rapporto più intermittente ..., ma che hanno saputo comunque regalarmi i momenti più divertenti (e di conseguenza speciali) della mia vita...con cui so di poter essere me stesso, anche se ho ancora seri dubbi su chi sia me stesso.
Siamo dei grandi <3.

Parole per persone vere, scritte da persone artificiali

— Written by G. Gemini, C. GPT and A. Claude

Padova, Aprile 2026

Luca Ribon

Indice

1. Introduzione	1.
1.1. Organizzazione del testo	1.
1.2. L'azienda	1.
1.3. Il progetto	1.
2. Descrizione dello stage	4.
2.1. Contesto aziendale	4.
2.2. Metodo di lavoro	4.
2.2.1. Versionamento e integrazione continua	4.
2.2.2. Monitoraggio	6.
2.2.3. Altri strumenti a supporto dello sviluppo	6.
2.3. Analisi dei rischi	7.
2.3.1. RA-01 - Variazione dei requisiti	7.
2.3.2. RA-02 - Scelte progettuali non adeguate	7.
2.3.3. RO-01 - Gestione del tempo non ottimale	7.
2.3.4. RT-01 - Inesperienza con tecnologie usate in azienda	7.
2.3.5. RT-02 - Influenze esterne	8.
2.3.6. RT-03 - Messaggi duplicati	8.
2.3.7. RT-04 - Perdita di messaggi	8.
2.3.8. RT-05 - Sicurezza della comunicazione MQTT	8.
2.3.9. RT-06 - Debugging difficoltoso	8.
2.3.10. RT-07 - Errata configurazione dei lettori RFID	9.
2.3.11. RT-08 - Disconnessioni dei dispositivi fisici	9.
2.3.12. Specifica dei Rischi	9.
3. Analisi dei requisiti	12.
3.1. Panoramica delle funzionalità	12.
3.1.1. Gestione dei reader RFID	12.
3.1.2. Ricezione e visualizzazione dei cartellini letti dai reader	13.
3.1.3. Errori e notifiche all'utente	13.
3.2. Attori	13.
3.3. Casi d'uso	13.
UC1: Aggiunta di un reader RFID	13.
UC2: Aggiornamento dati del reader RFID	14.
UC3: Configurazione reader RFID	15.
UC3.1: Modifica della configurazione di sistema del reader RFID	15.
UC3.2: Modifica della modalità operativa del reader RFID	16.
UC4: Eliminazione reader RFID	16.
UC5: Download del certificato associato al reader	17.
UC6: Visualizzazione dei cartellini letti dal reader	18.
UC6.1: Visualizzazione dei cartellini processati	18.
UC6.2: Visualizzazione dei cartellini scartati	19.
UC7: Errore: sistema non raggiungibile	19.

UC8: Errore: servizi cloud esterni non raggiungibili	20.
UC9: Errore: configurazione non valida	20.
UC10: Errore: eliminazione già effettuata	21.
UC11: Errore: certificato già scaricato	21.
3.4. Requisiti	22.
3.4.1. Requisiti funzionali	23.
3.4.2. Requisiti di qualità	24.
3.4.3. Requisiti di vincolo	24.
4. Introduzione teorica	26.
4.1. Aspetti teorici rilevanti	26.
4.1.1. Protocolli di comunicazione	26.
4.1.2. MQTT	26.
4.1.3. Certificati per l'autenticazione dei dispositivi	28.
4.2. Strumenti scelti	28.
4.2.1. Hosting del broker MQTT	29.
4.2.2. Coda per la gestione dei messaggi in ingresso dai reader RFID	30.
4.2.3. Stack di sviluppo	30.
4.3. Stack tecnologico preesistente	31.
4.3.1. Librerie e framework PHP	31.
4.3.2. Altre tecnologie	32.
5. Architettura e progettazione	34.
5.1. Flusso del sistema	34.
5.1.1. RFID Reader	34.
5.1.2. Struttura dei topic	36.
5.1.3. AWS IoT Core	36.
5.1.4. AWS SQS	39.
5.1.5. KanbanBOX	40.
6. Codifica	42.
6.1. Design pattern utilizzati	42.
6.1.1. Repository	42.
6.1.2. Dependency Injection	42.
6.1.3. Factory	43.
6.1.4. Command	43.
6.1.5. Handler	43.
6.2. Gestione dei reader RFID	44.
6.2.1. AwsIotClientImplementation	44.
6.2.2. MqttClientAws	45.
6.2.3. SetupNewReaderOnAwsIot	49.
6.2.4. UpdateReaderOnAwsIot	51.
6.2.5. PfxHelper	52.
6.2.6. Reader	52.
6.2.7. DownloadReaderCertificateRow e RfidDownloadCertificate	59.
6.2.8. DeleteRfidReaderRow	62.
6.3. Ricezione dei tag RFID letti	65.
6.3.1. Worker	65.

6.3.2.	RfidEventsMessageSerializer	66.
6.3.3.	ReadTagEventsMessage	68.
6.3.4.	ReportEventsMessage e ReaderReportContainsHeartbeat	70.
6.3.5.	RfidEventsMessageHandler, UpdateLastHeartbeatOfTheReaderCommand e RfidScan	72.
7.	Verifica e validazione	77.
7.1.	Analisi statica del codice	77.
7.2.	Implementazione dei test di unità	78.
7.3.	Implementazione dei test di integrazione	81.
7.4.	Continuous Integration	83.
8.	Conclusioni	86.
8.1.	Consuntivo finale	86.
8.2.	Conoscenze acquisite	86.
8.3.	Sviluppi futuri	86.
8.3.1.	Rotazione certificati	86.
8.3.2.	Configurazione di un dominio custom per il broker MQTT	87.
8.3.3.	Approfondimento della configurazione della coda SQS	87.
8.3.4.	Test di integrazione	87.
8.4.	Considerazioni finali	88.
	Bibliografia	90.
	Glossario	92
A		92
API		92
B		92
Backend		92
Backup		92
Broker		92
C		92
CLI		92
D		92
DTO		92
E		93
F		93
G		93
H		93
HTTP		93
IaaS		93
I		93
IDE		93
J		93
JSON		93
K		93
Kanban		93
L		93

Lean	93
M	94
MQTT	94
N	94
O	94
P	94
PEM	94
PFX	94
Q	94
Quality of Service	94
R	94
RFID	94
RSSI	94
S	95
T	95
Topic	95
U	95
UML	95
V	95
W	95
Wildcard	95
X	95
Y	95
Z	95

Elenco delle Figure

Figura 1 Flusso del sistema	34.
Figura 2 Tabella di gestione dei reader RFID	53.
Figura 3 Bottone di aggiunta del reader RFID	54.
Figura 4 Form di aggiunta e modifica dei reader RFID	54.
Figura 5 Form di modifica dei reader RFID	57.
Figura 6 JSON schema per la configurazione dei reader RFID	57.
Figura 7 Row operation di download del certificato	59.

Elenco delle Tabelle

Tabella 1	Dettagli relativi al rischio RA-01	9.
Tabella 2	Dettagli relativi al rischio RA-02	9.
Tabella 3	Dettagli relativi al rischio RO-01	9.
Tabella 4	Dettagli relativi al rischio RT-01	10.
Tabella 5	Dettagli relativi al rischio RT-02	10.
Tabella 6	Dettagli relativi al rischio RT-03	10.
Tabella 7	Dettagli relativi al rischio RT-04	10.
Tabella 8	Dettagli relativi al rischio RT-05	10.
Tabella 9	Dettagli relativi al rischio RT-06	11.
Tabella 10	Dettagli relativi al rischio RT-07	11.
Tabella 11	Dettagli relativi al rischio RT-08	11.
Tabella 12	Tabella del tracciamento dei requisiti funzionali	23.
Tabella 13	Tabella del tracciamento dei requisiti di qualità	24.
Tabella 14	Tabella del tracciamento dei requisiti di vincolo	24.
Tabella 15	Struttura di un messaggio MQTT con dimensione dei campi	27.
Tabella 16	Struttura di una richiesta HTTP con dimensione dei campi	28.

Capitolo 1.

Introduzione

1.1. Organizzazione del testo

Il documento è strutturato in sei capitoli principali:

1. **Introduzione**: describe.
2. **Descrizione dello stage**: describe.
3. **Analisi dei requisiti**: describe.
4. **Introduzione teorica**: describe.
5. **Descrizione del lavoro svolto**: describe.
6. **Conclusioni**: describe.

Riguardo la stesura del testo, sono state adottate le seguenti convenzioni tipografiche:

- gli acronimi, le abbreviazioni e i termini ambigui o di uso non comune vengono definiti nel glossario, situato alla fine del presente documento; per la prima occorrenza dei termini riportati nel glossario viene utilizzato il seguente stile: termine_G ;
- i termini in lingua straniera o facenti parti del gergo tecnico sono evidenziati con il carattere *corsivo*.

1.2. L'azienda

KanbanBOX S.r.l. è un'azienda con sede a Vicenza che si occupa dello sviluppo, della commercializzazione e della consulenza relativi alla piattaforma web KanbanBOX.

KanbanBOX è uno strumento progettato per supportare le aziende nell'implementazione di metodologie Lean_G nella gestione della produzione e della logistica delle aziende. Più nello specifico, la piattaforma consente di tracciare e gestire lo stato dei materiali grezzi, semilavorati e finiti, durante le fasi di approvvigionamento e produzione. Ciò è reso possibile attraverso l'utilizzo di cartellini Kanban_G elettronici, che permettono di identificare e tracciare i prodotti man mano che vengono consumati o resi disponibili dai processi aziendali.

1.3. Il progetto

KanbanBOX prevede due metodi di lettura dei cartellini Kanban:

- scansione di **codici a barre**: per ogni cartellino vengono stampata un'etichetta su cui sono presenti dei codici a barre che possono essere scansionati tramite appositi lettori ottici o tramite la fotocamera di comuni dispositivi mobili;
- lettura di chip **RFID**: per ogni cartellino viene stampata un'etichetta RFID, ovvero un'etichetta dotata di un *chip* che può essere letto da apposite antenne RFID.

In entrambi i casi le operazioni svolte sui cartellini vengono comunicate a KanbanBOX tramite delle API_G RESTful che sfruttano il protocollo HTTP per la trasmissione dei dati. In particolare, nel caso della lettura tramite RFID, il lettore RFID mette a disposizione delle API HTTP, eseguite in locale sul lettore stesso, che consentono la

comunicazione con esso.

I modelli di lettori RFID utilizzati da KanbanBOX supportano anche la comunicazione tramite protocollo MQTT, un protocollo di comunicazione del livello applicativo, basato su un meccanismo *publish/subscribe*; in aggiunta i lettori implementano nativamente un metodo di connessione sicuro e pratico al servizio AWS IoT Core.

MQTT è particolarmente adatto per la comunicazione con dispositivi *IoT*, in quanto progettato per essere meno oneroso in termini di consumo di banda e risorse computazionali rispetto a HTTP; inoltre prevede dei meccanismi di *Quality of Service (QoS)* e di memorizzazione dei messaggi che lo rendono più affidabile in scenari in cui la connettività di rete non è necessariamente stabile.

Per questi motivi il progetto di stage si è focalizzato sulla sostituzione del protocollo HTTP con MQTT per la trasmissione dei tag letti tra lettori RFID e KanbanBOX con l'obiettivo di migliorare l'affidabilità e l'efficienza della comunicazione; inoltre è stata prevista l'implementazione di funzionalità di configurazione dei lettori direttamente all'interno di KanbanBOX, operazioni che in precedenza dovevano essere eseguite manualmente tramite l'interfaccia web fornita dal produttore del lettore.

Ho scelto questo progetto perché prometteva di approfondire e di maturare esperienza con temi e tecnologie che mi incuriosiscono, come l'*Internet of Things*, i protocolli di comunicazione e le infrastrutture *cloud*. Un altro fattore interessante è stato il contatto diretto e concreto con il prodotto su cui ho lavorato che ho potuto vedere all'opera in un contesto reali.

Capitolo 2.

Descrizione dello stage

2.1. Contesto aziendale

Lo stage si è svolto in presenza presso la sede di Vicenza di KanbanBOX S.r.l. Sono stato inserito nel team sviluppatori software con il quale ho, fin da subito, partecipato a tutte le attività di routine previste per tutto il gruppo.

Per tutta la durata dello stage ho avuto a disposizione il supporto del mio tutor aziendale e del mio *buddy*, i quali mi hanno guidato nell’inserimento all’interno dell’azienda, del team e del prodotto KanbanBOX su cui ho lavorato.

Inoltre, ho avuto modo di collaborare anche con altri membri del team per discutere con chi aveva maggiore esperienza in merito agli argomenti o alle tecnologie con cui ero meno familiare.

Oltre al supporto *peer-to-peer* ricevuto ho potuto seguire diversi corsi di formazione interna organizzati dall’azienda per approfondire i concetti, sia teorici che pratici, legati alla metodologia Lean, ai cartellini Kanban e all’utilizzo di KanbanBOX.

2.2. Metodo di lavoro

Per il team di sviluppo sono previste diverse occasioni ricorrenti per la pianificazione e la revisione delle attività svolte:

- **Daily stand-up meeting:** riunione giornaliera di circa 15 minuti in cui ogni membro del team condivide lo stato di avanzamento del proprio lavoro, le difficoltà incontrate e gli obiettivi per la giornata corrente;
- **Dev+ weekly:** riunione settimanale del team di sviluppo a cui partecipa anche un consulente esterno, con esperienza rilevante nel mondo PHP *open source*, con cui si affrontano i problemi più ostici e si analizzano approfonditamente potenziali soluzioni;
- **Analisi issue Sentry:** riunione settimanale tra tutti i membri del team in cui si analizza il report delle *issue* tracciate da Sentry, uno strumento di monitoraggio degli errori in produzione; l’obiettivo dell’incontro è quello di identificare i bug più critici e frequenti e di assegnarli ai membri del team per la risoluzione;
- **OKR (Objective and Key Results):** riunione svolta su base trimestrale in cui, partendo da degli obiettivi aziendali più generici, si definiscono gli obiettivi specifici per il team di sviluppo e i risultati chiave che ne misurano il raggiungimento. Così facendo si assicura che ogni team si impegni a supportare gli obiettivi aziendali e non solo le proprie esigenze interne o del cliente.

2.2.1. Versionamento e integrazione continua

Per il **versionamento** di tutta la *codebase* e la documentazione di KanbanBOX viene utilizzato **Git**, un sistema di controllo di versione distribuito.

I repository Git sono ospitati su **GitHub**, una piattaforma utilizzata per la gestione del codice sorgente e la collaborazione tra sviluppatori, che fornisce funzionalità aggiuntive,

oltre al semplice versionamento, usate dal team di KanbanBox come la gestione delle *issue*, le *pull request* con relative verifiche del codice e l'integrazione continua.

Nel repository di KanbanBOX, ovvero quello contenente tutta la *codebase* del prodotto, viene usato il flusso di lavoro **Git Flow**, che prevede l'utilizzo di due rami principali, **master** e **development**.

In **master** viene mantenuta sempre la versione stabile e in produzione, mentre in **development** viene costruito lo *snapshot* della prossima *release*. In aggiunta a questi due rami principali, ne esistono altri con scopi specifici che seguono la seguente nomenclatura:

- **feature/nome-feature**: rami usati per lo sviluppo di nuove funzionalità;
- **fix/nome-fix**: rami usati per la risoluzione di bug;
- **hotfix/nome-hotfix**: rami usati per la correzione di bug critici in produzione;
- **chore/nome-chore**: rami usati per attività di manutenzione o che in generale non apportano modifiche percepibili dall'utente finale (aggiornamenti di librerie, configurazioni, implementazione di funzionalità esistenti che non prevedono risoluzione di bug).

Quando lo scopo di un ramo viene assolto interamente, viene aperta una **pull request** su GitHub per richiedere la revisione e l'integrazione delle modifiche nel ramo **development** (o **master** nel caso di hotfix).

Ad ogni pull request si applicano delle **politiche di revisione** in base all'impatto che l'*issue* risolta ha sul prodotto:

- *low*: nessuna revisione richiesta;
- *average*: revisione da parte di almeno un altro sviluppatore;
- *high*: revisione da parte di almeno due altri sviluppatori.

Quando si individuano problemi o nuove esigenze internamente al team vengono create delle *issue* su GitHub dai membri interessati seguendo un modello predefinito che ne facilita la categorizzazione e la prioritizzazione.

Oltre agli sviluppatori, anche il team dei consulenti applicativi utilizza i repository su GitHub per svolgere le attività di supporto ai clienti; infatti, ogni qualvolta viene aperta una richiesta di assistenza o di una nuova funzionalità da parte di un cliente, questa viene trasformata in un'*issue* su GitHub in cui vengono documentati i dettagli della richiesta ricevuta.

Infine le *issue* possono anche essere il risultato delle riunioni OKR.

L'assegnazione delle *issue* ai singoli membri avviene su base volontaria; ci sono però dei casi in cui l'assegnazione viene fatta in modo diretto, solitamente quando si hanno delle scadenze stringenti, così da essere sicuri di assegnare le *issue* ai membri con maggiore *ownership* tecnica sull'argomento trattato.

Inoltre, capita di scegliere volontariamente di mantenere certe *issue* a granularità meno fine, così che stia agli sviluppatori con più esperienza la decisione sul come suddividerle o se assegnarle ad un sotto-team dedicato.

Per ogni pull request aperta su GitHub vengono eseguite automaticamente le seguenti GitHub **Actions**:

- **PHPUnit**: esecuzione di test di unità, di integrazione ed *end-to-end*;
- **Playwright**: esecuzione di test *end-to-end* che simulano le operazioni utente tramite *browser* automatizzati;
- **PSALM**: analisi statica del codice PHP per individuare potenziali *bug* e problemi di sicurezza.

Tutti i test vengono eseguiti sia simulando il nuovo prodotto nella sua interezza, sia nelle seguenti casistiche meno comuni:

- esecuzione di nuove implementazioni o integrazioni nel codice in ambienti con le configurazioni della release stabile corrente; questo permette di individuare potenziali **problemi di retro-compatibilità**;
- esecuzione del vecchio codice con in ambienti con le nuove configurazioni previste per la prossima release; questo permette di misurare la capacità di fare **rollback** in caso di problemi con la nuova release.

Anche per i test vengono usati dei *container* in modo da isolare l'ambiente di esecuzione e garantire che i test siano più semplici da automatizzare e sempre riproducibili in modo consistente.

Giornalmente viene distribuita una **release** che include tutte le pull request approvate e integrate nel ramo **development**.

Rilevante è l'impegno che KanbanBOX sta impiegando per automatizzare il più possibile il processo di distribuzione, sfruttando, anche in questo caso, i Docker *container*.

2.2.2. Monitoraggio

Sentry è una piattaforma di **monitoraggio degli errori** che consente di tracciare, analizzare e risolvere i problemi nelle applicazioni in tempo reale.

All'interno di KanbanBOX viene utilizzata sia per ricevere notifiche automatiche sugli errori che si verificano in produzione, sia per gestire in modo più ordinato e collaborativo gli errori che si verificano aggiungendo dettagli utili ai *bug* su cui si sta lavorando e categorizzandoli.

Giornalmente viene scelto un membro del team di sviluppo che si dedicherà maggiormente a monitorare e analizzare le issue segnalate da Sentry in quella giornata. In più, settimanalmente viene organizzata una riunione dove il **responsabile Sentry** di quella giornata, assieme al resto del team, analizza e discute le issue più frequenti dell'ultimo periodo con l'obiettivo di categorizzarle, documentarle nel modo più dettagliato possibile e assegnarle ai membri del team per la risoluzione.

2.2.3. Altri strumenti a supporto dello sviluppo

Per lo sviluppo di KanbanBOX viene utilizzato **PhpStorm**, un **ambiente di sviluppo integrato (IDE_G)** specifico per il linguaggio PHP che offre funzionalità avanzate come l'analisi statica del codice, *refactoring* automatizzato, il *debugging* e l'integrazione con sistemi di controllo di versione come Git.

Tutti i servizi che compongono KanbanBOX, o che ne supportano lo sviluppo e il *testing*, sono eseguiti in dei *container*.

Docker è la piattaforma utilizzata per creare, distribuire e gestire questi container in modo efficiente; nel caso di KanbanBOX vengono utilizzati dei Dockerfile per istanziare e orchestrare i container, e dei file `docker-compose.yml` per definire la loro configurazione.

Per la **comunicazione** interna al team e con il resto dell'azienda viene utilizzato **Microsoft Teams**, una piattaforma di collaborazione che integra chat, videoconferenze, condivisione di file e integrazione con altre applicazioni Microsoft 365.

All'interno dell'azienda sono presenti sia canali generali per la comunicazione tra tutti i dipendenti, sia canali specifici per ogni team di lavoro. Inoltre sono presenti canali

dedicati ad attività specifiche come le riunioni Dev+ weekly e le analisi delle issue di Sentry (vedi §2.1.).

2.3. Analisi dei rischi

I rischi individuati e analizzati nella fase iniziale del progetto vengono identificati usando la seguente codifica:

R[categoria] - [numero]

Dove:

- **R:** indica che si tratta di un rischio;
- **[categoria]:** indica la categoria di appartenenza del rischio (A: di analisi e progettazione, O: organizzativo, T: tecnico);
- **[numero]:** indica il numero progressivo del rischio all'interno della categoria.

I rischi di analisi e progettazione derivano principalmente dal fatto che le tecnologie usate non sono ancora approfonditamente esplorate dal team di sviluppo, quindi è necessaria una fase di studio e sperimentazione per comprenderne al meglio le potenzialità, i limiti e le opzioni di integrazione con il prodotto.

2.3.1. RA-01 - Variazione dei requisiti

Descrizione: durante la fase di studio potrebbero emergere nuove informazioni che potrebbero essere incongruenti con le aspettative o rendere obsoleti i requisiti inizialmente definiti.

Soluzione preventiva: mantenere una comunicazione costante con il tutor aziendale per discutere eventuali cambiamenti nei requisiti.

2.3.2. RA-02 - Scelte progettuali non adeguate

Descrizione: durante la fase di integrazione e sviluppo potrebbero emergere nuove esigenze, limitazioni od opportunità date dalle tecnologie scelte.

Soluzione diagnostica: intervallare le fasi di integrazione o sviluppo con momenti di studio e sperimentazione mirati in modo da accertarsi che le scelte fatte siano adeguate, sfruttando anche le nuove informazioni derivanti dalle fasi di integrazione o sviluppo di altre tecnologie, svolte precedentemente.

2.3.3. RO-01 - Gestione del tempo non ottimale

Descrizione: durante lo svolgimento dello stage potrebbe non essere sempre possibile gestire il tempo in modo ottimale a causa di attività aziendali ausiliarie o imprevisti.

Soluzione preventiva: pianificazione intelligente degli appuntamenti di importanza secondaria in modo che non vadano a interferire con le attività principali dello stage.

2.3.4. RT-01 - Inesperienza con tecnologie usate in azienda

Descrizione: durante lo svolgimento dello stage potrebbero presentarsi dei rallentamenti dovuti alla mia inesperienza con alcune tecnologie o metodologie usate in azienda.

Soluzione preventiva e correttiva: dedicare la fase iniziale alla comprensione approfondita delle tecnologie ed interfacciarsi con i membri del team per chiedere supporto in caso di difficoltà bloccanti.

2.3.5. RT-02 - Influenze esterne

Descrizione: durante lo svolgimento dello stage potrebbero sorgere problemi tecnici dovuti a fattori esterni all'azienda, come ad esempio bug o disservizi di dipendenze esterne del prodotto.

Soluzione correttiva: analizzare il problema per identificare il prima possibile la reale causa, successivamente cercare di mitigarlo, se necessario coinvolgendo altri membri del team con maggiore esperienza; nel caso in cui il problema risultasse non risolvibile internamente ripianificare le attività in modo da minimizzare l'impatto sullo svolgimento dello stage.

2.3.6. RT-03 - Messaggi duplicati

Descrizione: durante la fase di fornitura potrebbero emergere problemi legati alla ricezione di messaggi MQTT duplicati causati da malfunzionamenti o configurazioni poco precise dei lettori RFID.

Soluzione preventiva:

- implementazione di un sistema di *debounce* lato backend di KanbanBOX che filtri i messaggi duplicati in base ad un identificativo univoco e ad una finestra temporale configurabile;
- configurazione dei lettori RFID per minimizzare la probabilità di letture duplicate, ad esempio regolando la potenza di trasmissione o la sensibilità dell'antenna.

2.3.7. RT-04 - Perdita di messaggi

Descrizione: durante la fase di fornitura potrebbero emergere problemi legati alla perdita di messaggi MQTT causati da malfunzionamenti della rete o dei servizi *middleware*.

Soluzione preventiva:

- configurazione del *Quality of Service (QoS)* di MQTT a livello 1 (ogni messaggio viene ricevuto almeno una volta) sia nei dispositivi che nel broker;
- configurazione della *message retention* nel broker MQTT in modo da conservare i messaggi non recapitati per un periodo di tempo configurabile.

2.3.8. RT-05 - Sicurezza della comunicazione MQTT

Descrizione: in fase di progettazione sono stati individuati dei potenziali rischi legati alla sicurezza della connessione e comunicazione tra i client del broker, data dall'utilizzo di un solo broker MQTT (integrato in AWS IoT) condiviso tra tutti i clienti.

Infatti, il broker di AWS IoT nella sua configurazione base permette a tutti i client di connettersi e comunicare tramite topic liberamente.

Soluzione preventiva: (vedi {reference a sezione dedicata} per una descrizione più dettagliata)

- utilizzo di certificati X.509 per l'autenticazione dei client MQTT, in modo da garantire che solo i client autorizzati possano connettersi e comunicare;
- creazione di policy dinamiche (che variano in base al dispositivo), basate su dei template, per limitare i permessi e l'accesso ai topic del dispositivo;
- utilizzo di topic granulari e specifici per ogni dispositivo, in modo da limitare la visibilità dei messaggi solo ai dispositivi autorizzati.

2.3.9. RT-06 - Debugging difficoltoso

Descrizione: durante la fase di sviluppo e testing potrebbero emergere difficoltà nel monitorare ed eseguire il *debug* i messaggi MQTT scambiati tra i dispositivi e il broker,

e successivamente tra la coda SQS e il backend di KanbanBOX.

Soluzione preventiva:

- sfruttare il sistema di *logging*, già presente nel progetto, nel nuovo codice sviluppato;
- usare strumenti di test o simulazione come il test client di AWS IoT o ElasticMQ per simulare una coda in locale.

2.3.10. RT-07 - Errata configurazione dei lettori RFID

Descrizione: una configurazione non corretta dei lettori RFID (endpoint MQTT, certificati, QoS, topic, modalità operativa) potrebbe causare malfunzionamenti nella trasmissione dei messaggi o comportamenti non coerenti tra dispositivi diversi.

Soluzione preventiva: fornire una configurazione standard funzionante che può essere facilmente usata e adattata dagli installatori.

2.3.11. RT-08 - Disconnessioni dei dispositivi fisici

Descrizione: disconnessioni frequenti o prolungate dei lettori RFID potrebbero causare interruzioni nella trasmissione dei dati e perdita di messaggi importanti.

Soluzione preventiva: implementare meccanismi di riconnessione automatica nei dispositivi, definendo parametri come il tempo di *keep-alive* e il numero di tentativi di riconnessione nei dispositivi.

2.3.12. Specifica dei Rischi

RA-01 - Variazione dei requisiti	
Tipologia	Analisi e progettazione
Probabilità	Media
Impatto	Alto

Tabella 1: Dettagli relativi al rischio RA-01

RA-02 - Scelte progettuali non adeguate	
Tipologia	Analisi e progettazione
Probabilità	Media
Impatto	Alto

Tabella 2: Dettagli relativi al rischio RA-02

RO-01 - Gestione del tempo non ottimale	
Tipologia	Organizzativo
Probabilità	Alta
Impatto	Alto

Tabella 3: Dettagli relativi al rischio RO-01

RT-01 - Inesperienza con tecnologie usate in azienda	
Tipologia	Tecnico
Probabilità	Alta
Impatto	Basso

Tabella 4: Dettagli relativi al rischio RT-01

RT-02 - Influenze esterne	
Tipologia	Tecnico
Probabilità	Bassa
Impatto	Alto

Tabella 5: Dettagli relativi al rischio RT-02

RT-03 - Messaggi duplicati	
Tipologia	Tecnico
Probabilità	Bassa
Impatto	Medio

Tabella 6: Dettagli relativi al rischio RT-03

RT-04 - Perdita di messaggi	
Tipologia	Tecnico
Probabilità	Bassa
Impatto	Alto

Tabella 7: Dettagli relativi al rischio RT-04

RT-05 - Sicurezza della comunicazione MQTT	
Tipologia	Tecnico
Probabilità	Bassa
Impatto	Alto

Tabella 8: Dettagli relativi al rischio RT-05

RT-06 - Debugging difficoltoso	
Tipologia	Tecnico
Probabilità	Alta
Impatto	Medio

Tabella 9: Dettagli relativi al rischio RT-06

RT-07 - Errata configurazione dei lettori RFID	
Tipologia	Tecnico
Probabilità	Bassa
Impatto	Alto

Tabella 10: Dettagli relativi al rischio RT-07

RT-08 - Disconnessioni dei dispositivi fisici	
Tipologia	Tecnico
Probabilità	Media
Impatto	Alto

Tabella 11: Dettagli relativi al rischio RT-08

Capitolo 3.

Analisi dei requisiti

3.1. Panoramica delle funzionalità

3.1.1. Gestione dei reader RFID

Dall'interfaccia di gestione dei reader RFID, strutturata come tabella, l'amministratore deve poter eseguire le seguenti operazioni:

- **Aggiunta** di un nuovo reader RFID, inserendo i seguenti dati:
 - *nome*: nome che aiuta l'utente a identificare il reader;
 - *produttore*: nome dell'azienda che ha prodotto il reader, attualmente sono disponibili solo reader prodotti da Zebra;
 - *modello*: nome o numero di modello del reader, attualmente sono disponibili solo reader della serie FX7500 e FX9600;
 - *indirizzo MAC*: indirizzo MAC del reader; campo opzionale poiché viene usato uno UUID generato dal backend di KanbanBox che risulta più affidabile, in quanto il MAC address può essere facilmente manipolato da chiunque abbia accesso al reader;
 - *abilitazione*: indica se il reader è abilitato e quindi utilizzabile o meno, permette di rendere temporaneamente inutilizzabile un reader senza doverlo eliminare e riconfigurare successivamente.

L'aggiunta deve scaturire sia la registrazione del reader nel database di KanbanBox, che la creazione delle entità corrispondenti al reader e al certificato in Aws IoT.

- **Aggiornamento** dei dati di un reader già registrato nel sistema, con parametri e modalità coerenti con la procedura di aggiunta;
- **Configurazione**
 - del **sistema**: permette di configurare i parametri per la connessione al broker MQTT, in questo caso integrato in AWS IoT Core, tra cui anche i certificati per autenticare il reader durante la comunicazione con AWS IoT;
 - della **modalità operativa**: e i parametri della comunicazione tramite MQTT (ad esempio topic o Quality of Service);
 - **modalità operativa**: permette di determinare i parametri di lettura dei tag RFID, come ad esempio la frequenza di rilettura, il batching e la potenza dell'antenna radio.

Le configurazioni, una volta applicate, dovranno essere salvate a database per poterle modificare più rapidamente in futuro.

- **Eliminazione** di un reader, che comporta anche l'eliminazione delle entità corrispondenti al reader e al certificato in Aws IoT;
- **Download del certificato** associato al reader, necessario per la comunicazione con AWS IoT, che può essere scaricato una sola volta per motivi di sicurezza. Nello specifico il certificato deve essere facilmente fruibile per utenti con meno dimestichezza come i consulenti che si occupano di configurare i reader presso i clienti.

3.1.2. Ricezione e visualizzazione dei cartellini letti dai reader

La piattaforma prevede già un'interfaccia per visualizzare i cartellini letti dai reader, con elementi grafici intuitivi e individuabili in modo chiaro, che permette di distinguere facilmente i cartellini processati da quelli scartati anche in ambienti meno agevoli come le linee di produzione delle aziende clienti.

La migrazione da HTTP a MQTT comporta però un'incompatibilità nella comunicazione tra i reader e KanbanBox; è quindi necessario **adattare l'implementazione del *pull*** dei cartellini da AWS SQS e dell'**interpretazione** dei dati trasmessi attraverso i tag letti, in modo che questi rimangano associabili ai cartellini kanban e al loro cambio stato, e che continuino ad essere distinti tra cartellini processati e scartati.

3.1.3. Errori e notifiche all'utente

In caso di **errori**, il sistema deve notificare all'amministratore l'impossibilità di completare l'operazione richiesta e l'errore riscontrato tramite dei **pop-up** a scomparsa, mostrati direttamente nella pagina in cui l'utente sta operando.

Il contenuto dei messaggi di errore deve racchiudere una breve descrizione dell'errore riscontrato, comprensibile anche per utenti non tecnici; inoltre, ove possibile, deve fornire dettagli utili a comprendere il problema, come ad esempio l'identificativo della risorsa su cui si stava operando.

3.2. Attori

Amministratore: nel backend di KanbanBOX sono presenti diversi ruoli utilizzati per definire i permessi all'interno della piattaforma. Nonostante ciò, le funzionalità implementate durante questo progetto di tesi sono accessibili in egual modo dai ruoli sviluppatore, amministratore o consulente applicativo, poiché è necessario che possano essere utilizzate da qualsiasi figura tecnica che si occupi di sviluppo o manutenzione della piattaforma.

Per questo motivo, in questa sede, ho deciso di raggruppare tutti i ruoli al di sotto dell'attore «Amministratore».

3.3. Casi d'uso

In questa sezione sono presenti i diagrammi UML_G che rappresentano i casi d'uso principali del sistema e le loro descrizioni.

Il progetto è focalizzato principalmente sulla creazione di un driver di interfacciamento, ovvero un componente software che si occupa di gestire la comunicazione tra i reader RFID, AWS IoT Core e la piattaforma web KanbanBox, e sulla progettazione dell'architettura del sistema che include reader, servizi AWS e backend di KanbanBox. Inoltre molte delle funzionalità direttamente o indirettamente legate alle modifiche o aggiunte fatte erano già implementate in origine e sono rimaste invariate e funzionanti. Per questo i casi d'uso relativi al progetto sono in numero ridotto e semplici.

UC1: Aggiunta di un reader RFID

Attore principale Amministratore

Descrizione	Il sistema consente all'amministratore di aggiungere un nuovo reader RFID alla piattaforma, configurando i parametri necessari per la comunicazione con AWS IoT.
Precondizioni	• L'amministratore deve essere autenticato; • Il reader RFID è configurato per poter comunicare in rete e con AWS IoT.
Postcondizioni	• Il reader RFID viene registrato nella piattaforma KanbanBox; • Il reader RFID viene registrato nei servizi cloud esterni.
Scenario principale	• L'amministratore inserisce i dati del reader RFID negli appositi campi: ▸ nome; ▸ produttore; ▸ modello; ▸ indirizzo MAC; ▸ abilitazione. Tutti i campi sono configurati in modo da accettare solo dati nel formato valido; • L'amministratore invia i dati del reader tramite il pulsante di salvataggio.
Estensioni	• Errore: sistema non raggiungibile; • Errore: servizi cloud esterni non raggiungibili.

UC2: Aggiornamento dati del reader RFID

Attore principale	Amministratore
Descrizione	Il sistema consente all'amministratore di aggiornare i dati di un reader RFID già registrato nella piattaforma.
Precondizioni	• L'amministratore deve essere autenticato; • Il reader RFID è configurato per poter comunicare in rete e con AWS IoT; • Il reader RFID è già registrato nella piattaforma KanbanBox.
Postcondizioni	• Il sistema ha aggiornato i dati del reader RFID nella piattaforma KanbanBox.
Scenario principale	• L'amministratore, dalla tabella dei reader, preme il bottone per modificare i dati del reader scelto; • L'amministratore inserisce i dati aggiornati del reader RFID negli appositi campi, configurati in modo da accettare solo dati nel formato valido: ▸ nome; ▸ produttore;

- modello;
- indirizzo MAC;
- abilitazione.
- L'amministratore invia le modifiche da applicare al reader tramite il pulsante di salvataggio.

Estensioni

- Errore: sistema non raggiungibile;
- Errore: servizi cloud esterni non raggiungibili.

UC3: Configurazione reader RFID

Attore principale	Amministratore
Descrizione	Il sistema consente all'amministratore di configurare un reader RFID già registrato nella piattaforma.
Precondizioni	• L'amministratore deve essere autenticato; • Il reader RFID è configurato per poter comunicare in rete e con AWS IoT; • Il reader RFID è già registrato nella piattaforma KanbanBox.
Postcondizioni	Il sistema ha aggiornato la configurazione di sistema e/o della modalità operativa del reader RFID e ne ha creato un <u>backup</u> salvato a database.
Scenario principale	• L'amministratore, dalla tabella dei reader, preme il bottone per configurare il reader scelto; • L'amministratore inserisce le configurazioni di sistema, della modalità operativa e dell'ambiente di comunicazione del reader, separatamente, negli appositi campi di testo; • L'amministratore invia le configurazioni del reader tramite il pulsante di salvataggio.
Estensioni	• Errore: sistema non raggiungibile; • Errore: servizi cloud esterni non raggiungibili.
Inclusioni	• Modifica della configurazione di sistema del reader RFID; • Modifica della modalità operativa del reader RFID.

UC3.1: Modifica della configurazione di sistema del reader RFID

Attore principale	Amministratore
Descrizione	Il sistema consente all'amministratore di modificare la configurazione di sistema del reader RFID, che include parametri utili per l'utilizzo di certificati di AWS IoT o la configurazione relativa a MQTT.
Precondizioni	• L'amministratore deve essere autenticato;

	• Il reader RFID è configurato per poter comunicare in rete e con AWS IoT; • Il reader RFID è già registrato nella piattaforma KanbanBox.
Postcondizioni	• L'amministratore ha inserito la configurazione di sistema nel formato apposito.
Scenario principale	• L'amministratore, dalla tabella dei reader, preme il bottone per configurare il reader scelto; • L'amministratore inserisce la configurazione di sistema del reader in formato JSON, che può anche essere generata tramite un form costruito basandosi su uno schema JSON apposito.
Estensioni	• Errore: configurazione non valida.

UC3.2: Modifica della modalità operativa del reader RFID

Attore principale	Amministratore
Descrizione	Il sistema consente all'amministratore di modificare la modalità operativa del reader RFID, che include parametri come la configurazione delle antenne radio, la frequenza di lettura dei tag, i dati inseriti nei messaggi MQTT e altri.
Precondizioni	• L'amministratore deve essere autenticato; • Il reader RFID è configurato per poter comunicare in rete e con AWS IoT; • Il reader RFID è già registrato nella piattaforma KanbanBox.
Postcondizioni	• L'amministratore ha inserito la configurazione della modalità operativa nel formato apposito.
Scenario principale	• L'amministratore, dalla tabella dei reader, preme il bottone per configurare il reader scelto; • L'amministratore inserisce la modalità operativa del reader in formato JSON, che può anche essere generata tramite un form costruito basandosi su uno schema JSON apposito.
Estensioni	• Errore: configurazione non valida.

UC4: Eliminazione reader RFID

Attore principale	Amministratore
Descrizione	Il sistema consente all'amministratore di eliminare un reader RFID già registrato nella piattaforma.
Precondizioni	• L'amministratore deve essere autenticato;

	• Il reader RFID è configurato per poter comunicare in rete e con AWS IoT; • Il reader RFID è registrato nella piattaforma KanbanBox.
Postcondizioni	• Il reader RFID viene eliminato dalla piattaforma KanbanBox; • Il reader RFID viene eliminato dai servizi cloud esterni.
Scenario principale	• L'amministratore, dalla tabella dei reader, preme il bottone per eliminare il reader scelto; • Tramite un pop-up viene chiesta conferma dell'eliminazione del reader RFID; • Se viene confermata l'eliminazione, il sistema procede a eliminare il reader RFID dalla piattaforma KanbanBox e dai servizi cloud esterni.
Estensioni	• Errore: sistema non raggiungibile; • Errore: servizi cloud esterni non raggiungibili; • Errore: eliminazione già effettuata.

UC5: Download del certificato associato al reader

Attore principale	Amministratore
Descrizione	Il sistema consente all'amministratore di scaricare, solamente una volta, il certificato associato al reader RFID, necessario per la comunicazione con AWS IoT.
Precondizioni	• L'amministratore deve essere autenticato; • Il reader RFID è configurato per poter comunicare in rete e con AWS IoT; • Il reader RFID è già registrato nella piattaforma KanbanBox.
Postcondizioni	Viene fornito il file del certificato, che può essere scaricato in locale una sola volta per motivi di sicurezza.
Scenario principale	• L'amministratore, dalla tabella dei reader, preme il bottone per scaricare il certificato del reader scelto; • Viene mostrato un pop-up per notificare all'amministratore che il certificato può essere scaricato una sola volta e per richiedere la conferma dell'operazione; • Se viene confermata l'operazione, il sistema fornisce il file del certificato da scaricare in locale.
Estensioni	• Errore: sistema non raggiungibile; • Errore: servizi cloud esterni non raggiungibili; • Errore: certificato già scaricato.

UC6: Visualizzazione dei cartellini letti dal reader

Attore principale	Amministratore
Descrizione	Il sistema implementa una dashboard da cui è possibile vedere i cartellini letti dai reader e il loro stato.
Precondizioni	• L'amministratore deve essere autenticato; • Almeno un reader RFID è configurato per poter comunicare in rete e con AWS IoT; • I reader hanno letto almeno un cartellino.
Postcondizioni	Viene mostrata una dashboard con i cartellini letti dai reader e il loro stato.
Scenario principale	• L'amministratore accede alla dashboard dei cartellini letti dai reader; • Viene mostrata una tabella con i cartellini letti dai reader e il loro stato; • Ogni cartellino letto persiste nella dashboard per un periodo di tempo configurabile dall'utente, dopo il quale viene rimosso dalla visualizzazione; • Un cartellino può essere visualizzato nuovamente solo se uno dei reader attivi continua a leggerlo.
Generalizzazioni	• Visualizzazione dei cartellini processati; • Visualizzazione dei cartellini scartati.

UC6.1: Visualizzazione dei cartellini processati

Attore principale	Amministratore
Descrizione	Nella dashboard di visualizzazione dei cartellini letti è presente una colonna dedicata ai cartellini processati dalla piattaforma KanbanBox.
Precondizioni	• L'amministratore deve essere autenticato; • Almeno un reader RFID è configurato per poter comunicare in rete e con AWS IoT; • I reader hanno letto almeno un cartellino; • Almeno uno dei cartellini letti dai reader contiene un identificatore, del reader da cui è stato letto, corrispondente a quello di uno dei reader attivi.
Postcondizioni	Il cartellino processato viene mostrato nella colonna dedicata.
Scenario principale	• L'amministratore accede alla dashboard dei cartellini letti dai reader; • I cartellini che riportano un identificatore del reader da cui sono stati letti conforme, e che includono un codice del cartellino che trova corrispondenza tra i kanban nella

piattaforma vengono mostrati nella colonna dedicata ai cartellini processati.

UC6.2: Visualizzazione dei cartellini scartati

Attore principale	Amministratore
Descrizione	Nella dashboard di visualizzazione dei cartellini letti è presente una colonna dedicata ai cartellini scartati dalla piattaforma KanbanBox.
Precondizioni	• L'amministratore deve essere autenticato; • Almeno un reader RFID è configurato per poter comunicare in rete e con AWS IoT; • I reader hanno letto almeno un cartellino; • Almeno uno dei cartellini letti dai reader contiene un identificatore, del reader da cui è stato letto, incompleto o che non corrisponde a nessuno dei reader attivi.
Postcondizioni	Il cartellino scartato viene mostrato nella colonna dedicata.
Scenario principale	• L'amministratore accede alla dashboard dei cartellini letti dai reader; • I cartellini che riportano un identificatore del reader da cui sono stati letti non conforme, o che non includono un codice del cartellino che trova corrispondenza tra i kanban nella piattaforma vengono mostrati nella colonna dedicata ai cartellini scartati.

UC7: Errore: sistema non raggiungibile

Attore principale	Amministratore
Descrizione	Il <u>backend</u> di KanbanBOX non è raggiungibile a causa di un malfunzionamento del sistema o di problemi di rete. In questo caso, il sistema deve notificare all'amministratore l'impossibilità di completare l'operazione richiesta.
Precondizioni	• L'amministratore deve essere autenticato; • L'amministratore sta tentando di eseguire un'operazione che richiede la comunicazione con il backend di KanbanBOX.
Postcondizioni	• Il sistema mostra un messaggio di errore all'amministratore, indicando che l'operazione richiesta non può essere completata.
Scenario principale	• L'amministratore tenta di eseguire un'operazione che richiede la comunicazione con il backend di KanbanBOX; • Il sistema rileva che il backend di KanbanBOX non è raggiungibile;

- Il sistema mostra un messaggio di errore all'amministratore.

UC8: Errore: servizi cloud esterni non raggiungibili

Attore principale	Amministratore
Descrizione	I servizi cloud esterni, come AWS IoT, non sono raggiungibili a causa di un problema di configurazione o di un errore nel backend di KanbanBox. In questo caso, il sistema deve notificare all'amministratore l'impossibilità di inoltrare la richiesta al servizio cloud esterno e di completare l'operazione richiesta.
Precondizioni	• L'amministratore deve essere autenticato; • L'amministratore sta tentando di eseguire un'operazione che richiede la comunicazione con un servizio cloud esterno.
Postcondizioni	Il sistema mostra un messaggio di errore all'amministratore, indicando che l'operazione richiesta non può essere completata.
Scenario principale	• L'amministratore tenta di eseguire un'operazione che richiede la comunicazione con un servizio cloud esterno; • Il backend rileva che il servizio cloud esterno non è raggiungibile; • Il sistema mostra un messaggio di errore all'amministratore.

UC9: Errore: configurazione non valida

Attore principale	Amministratore
Descrizione	La configurazione inserita dall'amministratore per un reader RFID non è valida, a causa della mancanza di alcuni dati o del mancato rispetto dei vincoli imposti dallo schema di validazione.
Precondizioni	• L'amministratore deve essere autenticato; • L'amministratore ha tentato di configurare un reader RFID.
Postcondizioni	Il sistema mostra un messaggio di errore all'amministratore, indicando che la configurazione inserita non è valida e specificando i motivi dell'invalidità.
Scenario principale	• L'amministratore tenta di configurare un reader RFID; • Il sistema valida la configurazione inserita dall'amministratore; • Il sistema rileva che la configurazione non è valida;

- Il sistema mostra un messaggio di errore all'amministratore, indicando i motivi dell'invalidità della configurazione.

UC10: Errore: eliminazione già effettuata

Attore principale	Amministratore
Descrizione	Il sistema restituisce un messaggio di errore all'amministratore quando tenta di eliminare un reader RFID che è già stato eliminato.
Precondizioni	• L'amministratore deve essere autenticato; • L'amministratore ha tentato di eliminare un reader RFID; • Il reader RFID in questione è già stato eliminato dalla piattaforma KanbanBox e dai servizi cloud esterni.
Postcondizioni	Il sistema mostra un messaggio di errore all'amministratore, indicando che il reader RFID è già stato eliminato.
Scenario principale	• L'amministratore, dalla tabella dei reader, preme il bottone per eliminare il reader scelto; • Tramite un pop-up viene chiesta conferma dell'eliminazione del reader RFID; • Un altro utente conferma l'eliminazione del reader RFID; • L'amministratore conferma l'eliminazione del reader RFID; • Il sistema procede ad eseguire la richiesta dell'amministratore, ma rileva che il reader RFID è già stato eliminato; • Il sistema mostra un messaggio di errore all'amministratore, indicando che il reader RFID è già stato eliminato.

UC11: Errore: certificato già scaricato

Attore principale	Amministratore
Descrizione	Il sistema restituisce un messaggio di errore all'amministratore quando tenta di scaricare il certificato, associato ad un reader, che è già stato scaricato in precedenza, poiché per motivi di sicurezza il certificato può essere scaricato una sola volta.
Precondizioni	• L'amministratore deve essere autenticato; • L'amministratore ha tentato di scaricare il certificato associato ad un reader RFID; • Il certificato associato in questione è già stato scaricato in precedenza.

Postcondizioni	Il sistema mostra un messaggio di errore all'amministratore, indicando che il certificato è già stato scaricato.
Scenario principale	• L'amministratore, dalla tabella dei reader, preme il bottone per scaricare il certificato del reader scelto; • Viene mostrato un pop-up per notificare all'amministratore che il certificato può essere scaricato una sola volta e per richiedere la conferma dell'operazione; • Un altro utente conferma l'operazione di download del certificato; • L'amministratore conferma l'operazione di download del certificato; • Il sistema procede ad eseguire la richiesta dell'amministratore, ma rileva che il certificato è già stato scaricato; • Il sistema mostra un messaggio di errore all'amministratore, indicando che il certificato è già stato scaricato.

3.4. Requisiti

In questa sezione vengono elencati i requisiti dettati dal progetto. Essendo questo un progetto che ha richiesto un'ampia fase di esplorazione, i requisiti sono stati identificati in diversi momenti, partendo con un'analisi iniziale del piano di lavoro e, successivamente, approfondendo l'analisi attraverso riunioni interne con alcuni membri del team di sviluppo.

Ogni requisito è identificato da un codice alfanumerico costruito seguendo la struttura:

R - [numero] - [tipo] - [priorità]

dove:

- **[numero]**: numero progressivo che identifica il requisito tra quelli dello stesso tipo;
- **[tipo]**: indica se il requisito è
 - **F**: funzionale, indica una funzionalità che il sistema deve implementare;
 - **Q**: di qualità, indica una caratteristica di qualità che il sistema deve soddisfare, come ad esempio prestazioni, sicurezza, usabilità, ecc.;
 - **V**: di vincolo, indica una limitazione che il sistema deve rispettare, imposta dagli stakeholder o da fattori esterni, come ad esempio l'utilizzo di determinate tecnologie, l'aderenza a standard specifici, ecc.;
- **[priorità]**: indica l'importanza del requisito, che può essere
 - **O**: obbligatorio, ovvero che deve essere necessariamente soddisfatto perché si possa considerare il progetto completato;
 - **D**: desiderabile, ovvero che sarebbe preferibile soddisfare, ma non è essenziale per considerare il progetto completato;
 - **FA**: facoltativo, ovvero un requisito migliorativo che può essere soddisfatto se ci sono risorse e tempo sufficienti.

Nelle tabelle Tabella 12, Tabella 13 e Tabella 14 sono riassunti i requisiti, le fonti da cui sono stati ricavati e il loro stato di raggiungimento.

3.4.1. Requisiti funzionali

Requisito	Descrizione	Fonti	Stato
R-01-F-O	Sviluppo del driver di interfacciamento tra antenna RFID Zebra e AWS IoT Core	Piano di lavoro, UC3, UC3.1, UC3.2, UC6	Raggiunto
R-02-F-O	L'amministratore deve poter aggiungere un reader al backend di KanbanBox e ad AWS IoT	UC1	Raggiunto
R-03-F-O	L'amministratore deve poter aggiornare i dati di un reader già registrato nel sistema	UC2	Raggiunto
R-04-F-O	L'amministratore deve poter configurare il sistema e la modalità operativa di un reader già registrato nel sistema	UC3, UC3.1, UC3.2	Raggiunto
R-05-F-O	L'amministratore deve poter eliminare un reader già registrato nel sistema	UC4	Raggiunto
R-06-F-O	L'amministratore deve poter scaricare il certificato associato ad un reader, una sola volta	UC5	Raggiunto
R-07-F-O	Il sistema deve implementare una dashboard da cui è possibile vedere i cartellini letti dai reader e il loro stato	UC6, UC6.1, UC6.2	Raggiunto
R-08-F-O	Il sistema deve notificare all'amministratore l'impossibilità di completare l'operazione richiesta e l'errore riscontrato	UC7, UC8, UC9, UC10, UC11	Raggiunto

Tabella 12: Tabella del tracciamento dei requisiti funzionali

3.4.2. Requisiti di qualità

Requisito	Descrizione	Fonti	Stato
R-01-Q-O	Test e validazione funzionale del driver sviluppato	Piano di lavoro	Raggiunto parzialmente
R-02-Q-O	Implementazione di meccanismi atti a garantire la sicurezza della comunicazione	Riunioni interne	Raggiunto
R-03-Q-D	Ottimizzazione delle performance del driver e gestione avanzata degli errori e dei log	Piano di lavoro	Raggiunto parzialmente
R-04-Q-FA	Redazione della documentazione tecnica relativa all'architettura del progetto, configurazioni e utilizzo del driver	Piano di lavoro	Raggiunto
R-05-Q-FA	Sperimentazione di scenari avanzati di lettura multipla di tag RFID e gestione dei conflitti	Piano di lavoro	Raggiunto

Tabella 13: Tabella del tracciamento dei requisiti di qualità

3.4.3. Requisiti di vincolo

Requisito	Descrizione	Fonti	Stato
R-01-V-O	Utilizzo del protocollo MQTT per la comunicazione tra i reader RFID, AWS IoT Core e il backend di KanbanBox	Piano di lavoro, UC3, UC3.1, UC3.2	Raggiunto
R-02-V-O	Utilizzo di AWS IoT Core come servizio cloud per la gestione della comunicazione tra i reader RFID e il backend di KanbanBox	Piano di lavoro, UC6	Raggiunto
R-03-V-O	Utilizzo di AWS SQS come servizio cloud per la gestione delle letture dei tag RFID asincrona	Riunioni interne	Raggiunto
R-04-V-O	Utilizzo di PHP per l'implementazione delle funzionalità richieste	Riunioni interne	Raggiunto

Tabella 14: Tabella del tracciamento dei requisiti di vincolo

Capitolo 4.

Introduzione teorica

4.1. Aspetti teorici rilevanti

4.1.1. Protocolli di comunicazione

I **protocolli di comunicazione** sono parte integrante del progetto in quanto, come anticipato, uno degli obiettivi principali è quello di sostituire l'attuale protocollo HTTP con un protocollo più efficiente e adatto alla comunicazione tra dispositivi IoT, ovvero MQTT.

HTTP è il protocollo più diffuso per la comunicazione sul Web e costituisce la base delle **API** RESTful, sui cui si basava il metodo di comunicazione utilizzato nella precedentemente implementazione dei reader in KanbanBOX. Infatti i reader Zebra, una volta configurati, mettono a disposizione un'interfaccia che permette di inviare comandi e ricevere dati dei tag letti tramite richieste HTTP. Questa soluzione, sebbene semplice da implementare, presenta diverse limitazioni per il caso d'uso di KanbanBOX:

- **overhead elevato:** ogni richiesta HTTP include un *header* di dimensioni considerevoli rispetto al *payload* effettivo; in scenari come questo dove i lettori RFID devono inviare frequentemente piccoli pacchetti di dati, l'aumento di overhead su grandi quantità di messaggi inizia a diventare significativo;
- **assenza di comunicazione bidirezionale nativa:** HTTP non prevede un meccanismo nativo per cui il server possa inviare messaggi al client senza che quest'ultimo li richieda esplicitamente, rendendo complessa l'implementazione di operazioni come il *polling* dei tag letti dal reader;
- **nessuna garanzia di consegna:** HTTP non offre meccanismi integrati di *Quality of Service* per garantire la consegna dei messaggi in caso di disconnessioni temporanee, il che rappresenta un valore aggiunto in un contesto dove l'affidabilità dell'intero sistema è cruciale.

Queste limitazioni hanno motivato la migrazione a MQTT come protocollo per la comunicazione tra i lettori RFID e KanbanBOX.

4.1.2. MQTT

Come anticipato **MQTT** è stato scelto proprio per ovviare alle limitazioni che HTTP presenta in questo contesto. Infatti MQTT è protocollo di comunicazione progettato specificamente per la comunicazione tra dispositivi IoT, con **ottime performance** anche con risorse limitate, **affidabile** e adatto a gestire **grandi quantità di dispositivi e messaggi**.

L'architettura di MQTT si basa su un modello di ***publish/subscribe***, in cui i dispositivi (in questo caso i lettori RFID) pubblicano messaggi su specifici ***topic*** e altri dispositivi (come ad esempio il backend di KanbanBOX) si sottoscrivono a questi topic per ricevere i messaggi. Il tutto viene orchestrato da un ***broker*** MQTT, ovvero un server che gestisce la distribuzione dei messaggi tra i publisher e i subscriber tramite i topic definiti.

I topic in MQTT possono essere strutturati in modo **gerarchico**, dove ogni livello viene nominato con caratteri alfanumerici e separato da una barra («/»). Si possono anche utilizzare delle wildcard per sottoscrivere a più topic contemporaneamente.

Per ogni topic può essere definito un livello di **Quality of Service** (QoS) che determina con quale garanzia i messaggi vengono consegnati ai dispositivi sottoscritti, con tre livelli disponibili: 0 (al massimo una consegna), 1 (almeno una consegna) e 2 (esattamente una consegna). Inoltre, sempre con granularità a livello di topic, è possibile impostare la **retention** dei messaggi, ovvero un meccanismo dove il broker memorizza l'ultimo messaggio pubblicato su un topic e lo rende disponibile ai nuovi dispositivi iscritti al topic, così che tutti i dispositivi (compresi quelli appena iscritti) conoscano lo stato del topic anche dopo svariato tempo dall'ultima pubblicazione di un messaggio.

Diversi broker implementano anche la **persistence** dei messaggi, ovvero la memorizzazione dei messaggi su disco per garantire la loro conservazione anche in caso di malfunzionamenti software o hardware (che non intacchino il disco stesso) della macchina su cui il broker è in esecuzione.

Come anticipato i **pacchetti** (messaggi) di MQTT hanno un overhead più essenziale rispetto a quelli di HTTP, infatti la loro struttura è la seguente:

- **fixed header**: obbligatorio e di 2 *byte*, contiene informazioni di controllo come il tipo di messaggio (CONNECT, PUBLISH, SUBSCRIBE, ecc.), altre *flag* per il controllo di parametri come il livello di QoS o la retention del messaggio e il numero di byte rimanenti per quel messaggio;
- **variable header**: opzionale e di dimensione variabile, contiene informazioni aggiuntive come il protocollo e la versione utilizzati, o un identificatore univoco del pacchetto;
- **payload**: di dimensione variabile, contiene i dati effettivi del messaggio, che in questo caso saranno le letture dei tag RFID.

Quindi si parla di un overhead minimo di **~2 byte** per ogni messaggio MQTT, contro gli almeno **~85 byte** di overhead per ogni richiesta HTTP fatta dai reader.

Fixed Header (2 byte)			Variable Header (Dimensione variabile)	Payload (Dimensione variabile)
Packet Type (4 bit)	Flag (4 bit)	Remaining Length (Dimensione variabile, lunghezza del <i>variable header</i> e del <i>payload</i>)		

Tabella 15: Struttura di un messaggio MQTT con dimensione dei campi

Request Line		
Method (4 byte)	URI (Dimensione variabile)	HTTP Version (8 byte + 4 byte per il framing)
Headers		
Host (6byte per il nome del campo + ~7 byte per l'IP/hostname)	Content-Type (14 byte per il nome del campo + 16 byte per il valore del campo)	Content-Length (16 byte per il nome del campo + 1-5 byte per il valore del campo e il framing)
Empty Line ending headers (CRLF) (2 byte)		
Payload (Dimensione variabile)		

Tabella 16: Struttura di una richiesta HTTP con dimensione dei campi

4.1.3. Certificati per l'autenticazione dei dispositivi

Per garantire la sicurezza della comunicazione tra i reader RFID e il broker MQTT, è stato utilizzato il meccanismo di autenticazione basato su certificati *X.509*, integrato in AWS IoT Core.

Al momento della creazione di un'entità client su AWS IoT Core, la procedura di configurazione permette di scegliere se generare un **certificato** e le relative **chiavi crittografiche** associate a quello specifico client. Una volta generati i file, contenenti quelle che non sono altro che delle stringhe alfanumeriche, è possibile scaricarli in formato ***PEM***; l'operazione di download è disponibile solamente durante la fase di creazione del client, per policy di AWS atte a garantire la sicurezza della connessione al servizio.

Per rendere più pratica l'installazione dei reader RFID nelle aziende clienti si è deciso di gestire questi file sensibili interamente nel backend di KanbanBOX, in modo da fornire agli installatori, tramite l'interfaccia web, un unico file in formato ***PFX***, che incapsula i certificati e le chiavi fornite da AWS, ovvero l'unico formato accettato dai reader Zebra per il caricamento del certificato. In questo modo si evita agli installatori l'onere di dover compiere operazioni macchinose, per utenti che hanno meno dimestichezza con questi strumenti, e rischiose, in caso di fughe di dati.

Anche lato KanbanBOX i certificati rimangono scaricabili solamente una volta e successivamente vengono rimossi da qualsiasi dispositivo o servizio di archiviazione su cui erano stati memorizzati.

4.2. Strumenti scelti

In questa sezione vengono esposti i servizi e le tecnologie scelte per affrontare il progetto e i fattori che hanno portato a preferirli rispetto alle alternative disponibili.

4.2.1. Hosting del broker MQTT

Per l'hosting del broker MQTT e dei servizi a supporto della comunicazione tra i reader RFID e KanbanBOX si è optato fin da subito per una soluzione **cloud**. Questa scelta è stata dettata da diversi fattori: innanzitutto, l'azienda non disponeva di infrastruttura hardware adeguata a questo scopo; inoltre, uno degli obiettivi del progetto era quello di permettere la configurazione da **remoto** dei reader RFID tramite KanbanBOX. Questo richiede di accedere alle reti interne dei clienti (alle quali i reader si connettono per raggiungere la rete esterna) che sono tipicamente soggette a **regole di accesso molto restrittive**. Risulta quindi più agevole e sicuro richiedere ai clienti di abilitare il traffico verso i server di un servizio cloud riconosciuto e affidabile, piuttosto che verso i server locali di KanbanBOX.

Di conseguenza sono state valutate le opzioni di hosting in cloud disponibili sul mercato, prima di tutto confrontando i *provider* che offrono soluzioni adatte, tra cui Google Cloud, Microsoft Azure e **Amazon Web Services**. La scelta è ricaduta su AWS, principalmente per la già consolidata presenza di servizi AWS nell'infrastruttura di KanbanBOX, così da poter integrare la nuova infrastruttura con i servizi già in uso.

In concomitanza con la scelta del provider sono stati confrontati i servizi offerti dai provider sopra citati, in particolare, una volta indirizzatisi verso AWS, sono stati presi in considerazione AWS EC2 e AWS IoT Core.

AWS EC2 è un servizio di tipo *Infrastructure as a Service* che permette di istanziare macchine virtuali su cui è possibile installare e configurare qualsiasi software, generalmente con l'obiettivo di utilizzarlo come server; nel nostro caso, si sarebbe trattato di installare un broker MQTT (come Mosquitto) e configurarlo per gestire la comunicazione tra i reader RFID e KanbanBOX.

Anche **AWS IoT Core** è un servizio *Infrastructure as a Service*, ma è specificamente fornito per la gestione di dispositivi IoT, infatti include un broker MQTT e tutte le funzionalità a supporto come la gestione dei dispositivi o la configurazione di meccanismi di sicurezza specifici.

Il sottostante dei due servizi è molto simile, infatti entrambi si basano su un'infrastruttura EC2, ma AWS IoT Core presenta i seguenti **vantaggi** che hanno portato a preferirlo per questo progetto:

- **configurazione e gestione:** AWS IoT Core è progettato specificamente per la gestione di dispositivi IoT e questo porta a dei tempi e sforzi necessari per configurare e gestire il broker MQTT molto inferiori rispetto a quelli necessari con AWS EC2; inoltre, AWS IoT Core include delle **integrazioni** con servizi come AWS SQS che hanno facilitato molto l'implementazione di alcune funzionalità come la coda asincrona per la gestione dei messaggi in ingresso dai reader RFID;
- **costi:** AWS IoT Core prevede un modello di *pricing on demand*, ovvero pagato in base all'effettivo utilizzo del servizio, che viene misurato in messaggi trasmessi, dispositivi connessi, durata della connessione e altre metriche; basandosi su una stima di utilizzo per l'integrazione in KanbanBOX, sono stati confrontati i costi di AWS EC2 [1] e AWS IoT Core [2] e si è notato come la differenza non giustifichi le complicazioni date dall'utilizzo di AWS EC2; inoltre, in questo caso, il modello di pagamento *on demand* di EC2 risulta più difficile da gestire e quindi in periodi di utilizzo meno intenso c'è il rischio di pagare più del necessario, rispetto ad AWS IoT Core dove l'adeguamento al carico di lavoro è più flessibile e completamente automatizzato di *default*.

4.2.2. Coda per la gestione dei messaggi in ingresso dai reader RFID

Per la gestione dei messaggi in ingresso dai reader RFID, si è deciso di implementare una coda che permettesse di gestire i messaggi in modo più efficiente. In questo modo, i messaggi vengono inseriti in una coda e processati da un *worker* dedicato, implementato in KanbanBOX, che distribuisce i messaggi ai servizi di KanbanBOX adibiti al consumo dei messaggi.

In questo caso il confronto iniziale è stato svolto tra l'utilizzo della **coda integrata nel broker MQTT** di AWS IoT Core e l'integrazione di una coda dedicata.

La prima opzione, però, è stata scartata fin da subito poiché presentava delle limitazioni, infatti la coda integrata da AWS IoT può essere considerata una coda asincrona ma nella pratica non è altro che una *persistent session*, ovvero un meccanismo che permette di mantenere lo stato e i messaggi di un dispositivo anche in caso di disconnessione, ma che presenta le seguenti limitazioni:

- **durata massima della persistenza** dei messaggi in coda limitata ad un'ora;
- **numero massimo di messaggi** in coda limitato a 10 per ogni sottoscrizione ad un topic, con un limite complessivo di 100 messaggi in coda di cui non è stata confermata la ricezione per ogni dispositivo.

Inoltre, in KanbanBOX le code asincrone fornite da AWS erano già state utilizzate in altri domini della piattaforma, quindi il middleware per l'integrazione della coda era quasi interamente disponibile e testato, rendendo l'implementazione di una coda dedicata molto più semplice.

Per questo si è passati al confronto delle code asincrone proposte da AWS, ovvero **AWS SQS**, **AWS SNS** e **AWS MQ**.

- **AWS SQS** è un servizio di coda asincrona basata sul modello *pull*, in cui i messaggi vengono inseriti in una coda e i *consumer* devono interrogare la coda per ricevere i messaggi; ha dei costi più bassi relativamente alle altre opzioni ed è quella con le capacità di scalabilità più elevate;
- **AWS SNS** è un servizio di messaggistica basato sul modello *push*, in cui i messaggi vengono pubblicati su un topic e i *subscriber* ricevono i messaggi in tempo reale; è più costoso di AWS SQS;
- **AWS MQ** è un servizio di messaggistica basato su Apache ActiveMQ o RabbitMQ; è più costoso e meno scalabile di AWS SQS e AWS SNS, inoltre viene suggerito per scenari in cui è necessario migrare da un'infrastruttura *on-premises* a una in cloud mantenendo la compatibilità con protocolli di messaggistica standard (come AMQP o MQTT).

Alla luce di queste considerazioni, **AWS SQS** è risultato essere la scelta più adatta per questo progetto, principalmente per il suo modello di funzionamento *pull* che si adatta meglio al caso d'uso di KanbanBOX, dove vogliamo che sia il worker da noi implementato a gestire il *polling* dei messaggi in coda.

4.2.3. Stack di sviluppo

Per l'implementazione in PHP del driver di comunicazione MQTT e l'integrazione con i servizi di AWS ci si è affidati a delle librerie *open source* di PHP e ad alcuni SDK ufficiali di AWS.

In particolare abbiamo **php-mqtt** ([3]), una libreria *open source* che permette di implementare un client MQTT in PHP; include classi, tra cui alcuni *DTO_G*, e metodi che facilitano l'implementazione delle seguenti funzionalità:

- connessione ad un broker MQTT;
- gestione delle impostazioni di connessione al broker MQTT;
- pubblicazione di un messaggio su un topic MQTT;
- sottoscrizione a un topic MQTT e ricezione dei messaggi pubblicati su quel topic;
- utilizzo di TLS per crittografare la comunicazione tra il client e il broker MQTT;
- meccanismi di QoS e retention dei messaggi MQTT.

Le alternative disponibili sono poche, tra le più rilevanti abbiamo **phpMQTT** ([4]) e **simps-mqtt** ([5]), che però risultano nettamente inferiori rispetto a **php-mqtt**; infatti la prima è una libreria non più mantenuta da diversi anni, con un numero di funzionalità molto limitato e una documentazione quasi inesistente, mentre la seconda è una libreria più recente ma ancora acerba, con un numero di funzionalità limitato e una documentazione superficiale.

Dall'altra parte, invece, **php-mqtt** è una libreria **mantenuta** con costanza, più **curata** delle altre due, con un numero di funzionalità più ampio e una documentazione più completa, che include anche esempi di utilizzo per le funzionalità più rilevanti.

L'**SDK ufficiale di AWS IoT** ([6]), nella sua versione per PHP, è stato utilizzato per manipolare tutte le entità di AWS IoT Core, utili per rappresentare i client lato AWS, per gestire i certificati e per l'instradamento dei messaggi.

Mentre l'**SDK ufficiale di AWS SQS** ([7]), sempre nella sua versione per PHP, è stato utilizzato per interagire con il servizio di coda asincrona di AWS, in particolare per leggere i messaggi dalla coda.

Nel caso degli SDK di AWS l'unica alternativa plausibile sarebbe stata quella di utilizzare la *CLI_G* di AWS, lanciando i comandi tramite PHP, ma è stata scartata fin da subito in quanto non presenta alcun vantaggio rispetto all'utilizzo degli SDK ufficiali e, anzi, avrebbe reso l'implementazione molto più ostica.

4.3. Stack tecnologico preesistente

In questa sezione vengono elencati e descritti gli strumenti e le tecnologie già utilizzati in azienda e che sono stati sfruttati durante l'implementazione esposta in questo documento.

4.3.1. Librerie e framework PHP

Framework e Core:

- **CodeIgniter:** framework MVC utilizzato come architettura di base nel codice legacy di KanbanBOX, che è ancora in fase di migrazione verso un'architettura più moderna basata su Symfony;
- **Symfony:** framework PHP moderno verso cui si sta migrando in KanbanBOX; fornisce un'architettura modulare e *component-based*, con un ecosistema ricco di librerie che facilitano lo sviluppo di applicazioni web complesse e scalabili;
- **Doctrine:** è un Object-Relational Mapper (ORM), ovvero una libreria che permette di mappare le entità del dominio di KanbanBOX a tabelle di un database relazionale, facilitando la gestione della persistenza dei dati e l'astrazione del database;

- **Twig:** *template engine* performante e flessibile utilizzato per la generazione di interfacce web dinamiche; permette di definire dei template HTML con una sintassi specifica e più intuitiva, i template vengono poi utilizzati nella logica di generazione delle pagine web di KanbanBOX.

Standard e Utility:

- **Composer:** gestore di dipendenze standard de facto per PHP, utilizzato per l'installazione, aggiornamento e integrazione di librerie esterne e per la gestione dell'*autoloading* delle classi;
- **Guzzle:** client HTTP utilizzato per l'interazione con servizi web esterni e API RESTful; in KanbanBOX viene utilizzato anche per gestire richieste tra endpoint interni diversi, ad esempio per il download dei certificati dei reader;
- **Psr (PHP Standard Recommendations):** libreria che definisce una serie di standard e mette a disposizione interfacce per garantire l'interoperabilità tra le librerie e i framework PHP, facilitando l'integrazione di componenti di terze parti e promuovendo un'architettura modulare e scalabile;
- **Psl (PHP Standard Library):** libreria standard che aiuta ad imporre un approccio fortemente tipizzato e orientato agli oggetti nello sviluppo di applicazioni PHP;
- **ramsey/UUID:** libreria per la generazione di UUID (Universally Unique Identifier), utilizzata per creare identificatori univoci per le entità del dominio di KanbanBOX, come ad esempio i reader RFID o i tag RFID letti;
- **i18next:** libreria per la gestione dell'internazionalizzazione (i18n), utilizzata per supportare più lingue nell'interfaccia di KanbanBOX;
- **openssl:** estensione di PHP che fornisce funzioni per la crittografia e la generazione di certificati, utilizzata per la generazione dei file PFX da fornire agli installatori dei reader RFID.

Testing e Qualità: tutti i controlli elencati di seguito vengono eseguiti tramite l'integrazione continua configurata su **GitHub Actions**, che permette di eseguire i test e le analisi di qualità del codice per ogni pull request.

- **PHPUnit:** framework principale per l'esecuzione di *unit test*, ovvero che permette di testare l'unità di codice minima nel progetto, fondamentale per validare la *business logic*;
- **Playwright:** tool di automazione per il testing *end-to-end*, utilizzato per simulare le interazioni dell'utente nel browser e verificare che il risultato sia quello previsto in fase di progettazione;
- **Infection:** strumento di *mutation testing* che verifica la robustezza dei test implementati, introducendo modifiche al codice sorgente originale.
- **PSALM:** analisi statica del codice PHP utile a individuare bug e problematiche potenziali prima di rilasciare le modifiche in produzione.

4.3.2. Altre tecnologie

- **MySQL:** database relazionale utilizzato per la persistenza dei dati in KanbanBOX;
- **Make:** utility di automazione utile per gestire l'intero ciclo di vita del software, dalla fase di sviluppo a quella di testing e deployment, semplificando l'esecuzione di comandi complessi e ripetitivi che vengono raggruppati e mappati in comandi più semplici tramite un file di configurazione detto Makefile;

- **AWS IAM:** servizio di gestione degli accessi di Amazon Web Services, utilizzato per configurare i ruoli e i relativi permessi di accesso alle varie funzionalità di AWS da parte dei servizi e degli sviluppatori di KanbanBOX;
 - **Reader Zebra:** dispositivi hardware prodotti dal Zebra Technologies; nel caso di questo progetto si ha interagito principalmente con i reader RFID della serie FX7500 e FX9600 ma in KanbanBOX sono ampiamente utilizzati anche dispositivi come barcode scanner e stampanti di etichette.
- Nello specifico i reader hanno il compito di leggere i tag RFID in prossimità, tramite delle antenne esterne, e di trasmettere i dati dei tag letti agli endpoint configurati.

Capitolo 5.

Architettura e progettazione

5.1. Flusso del sistema

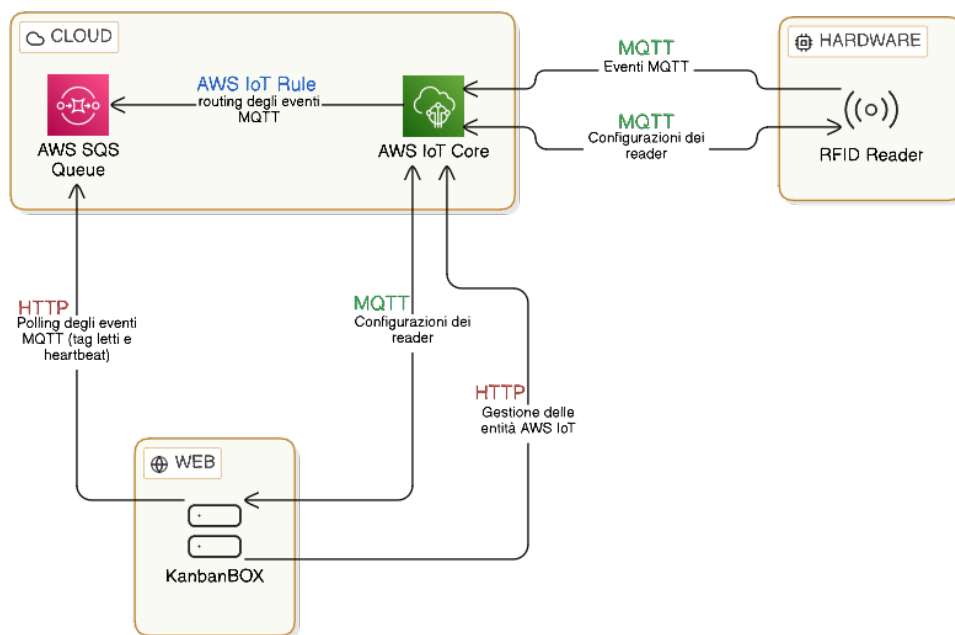


Figura 1: Flusso del sistema

5.1.1. RFID Reader

I reader RFID Zebra sono dei dispositivi hardware che, se muniti di uno o più moduli esterni che fungono da antenne, sono in grado di leggere i tag RFID presenti nell'ambiente circostante.

Per poter trasmettere dati e comandi, da e verso il reader, è necessario configurare la connessione ad un endpoint esterno; nel nostro caso, in cui la scelta del broker MQTT è ricaduta su AWS IoT Core, è stato necessario configurare un endpoint di tipo **AWS IoT Connector**, ovvero un'interfaccia implementata da Zebra nei propri reader che consente di farli comunicare con AWS IoT Core tramite MQTT, configurando un numero ridotto di parametri.

La configurazione, infatti, richiede di definire:

- il *domain name* di AWS IoT Core;
- la porta da utilizzare per la trasmissione;
- l'identificativo che si vuole usare per il dispositivo che si sta configurando;

- il certificato, già memorizzato nel reader, da utilizzare per l'autenticazione con AWS IoT Core e la relativa *passphrase*.

Inoltre i reader Zebra permettono di definire quale topic utilizzare per le seguenti macro-categorie di messaggi:

- **management events**: utilizzato principalmente come log di sistema e messaggi di diagnostica, nel nostro caso è utilizzato principalmente per trasmettere l'**heartbeat** del reader, ovvero un messaggio periodico che contiene diversi parametri di stato del reader;
- **tag data events**: utilizzato per trasmettere i dati dei tag RFID letti dal reader, è il topic più importante per il nostro sistema in quanto contiene i dati che ci interessano per la gestione dei **kanban**;
- **management**:
 - *command*: comandi di gestione del sistema, aggiornamenti, configurazione dell'endpoint e della connessione, ecc.;
 - *response*: risposte ai comandi sopra elencati che comunicano il successo o il fallimento e il relativo messaggio di errore;
- **control**:
 - *command*: comandi di controllo della lettura dei tag, quindi configurazione delle antenne come potenza, modalità di lettura, ecc.;
 - *response*: risposte ai comandi di controllo sopra elencati che comunicano il successo o il fallimento e il relativo messaggio di errore.

Una volta configurati tutti i parametri elencati il reader potrà connettersi al broker MQTT integrato in AWS IoT Core e iniziare a trasmettere i messaggi in base alla configurazione dei topic sopra descritta.

Oltre ai parametri di connessione, anche la modalità di lettura (detta **modalità operativa**) è configurabile tramite un'apposita interfaccia web in esecuzione sui reader stessi, e quindi raggiungibile collegandosi direttamente all'IP del reader tramite un browser web; in alternativa è possibile configurare i parametri sopracitati tramite dei comandi MQTT, inviati su appositi topic configurati nel reader, che seguono uno schema JSON definito da Zebra. Per poter configurare il reader (sia i parametri di connessione che la modalità operativa) **direttamente dall'interfaccia di KanbanBOX** abbiamo deciso di sfruttare i comandi MQTT, usando lo stesso flusso di dati MQTT utilizzato per trasmettere messaggi dei tag e di diagnostica, per questo è necessario poter trasmettere messaggi MQTT anche dal backend di KanbanBOX verso i reader, passando per AWS IoT Core. Per adempiere a questo requisito è stata sfruttata la libreria **php-mqtt** ([3]).

Nel dominio di KanbanBOX ogni reader RFID può essere associato ad una o più **aree**; ogni area rappresenta un insieme di una o più **antenne RFID**. In questo modo si riescono a rappresentare, dal punto di vista della logica di dominio, le antenne fisiche installate nelle linee di produzione che vengono collegate ai reader RFID.

Inoltre ad ogni area viene associato un **cambio stato** [8] del cartellino, di conseguenza, nella logica di dominio, basterà associare l'antenna che ha eseguito la lettura all'area corrispondente (tramite le relazioni definite a DB) per poter associare le letture dei tag ad un cambio stato del cartellino kanban.

5.1.2. Struttura dei topic

La struttura dei topic è stata **standardizzata** in modo da poterla replicare per tutti i reader senza incorrere in problemi di incongruenza tra le varie configurazioni.

Nei primi livelli della struttura sono stati sfruttati nome del **produttore** e del **modello** del dispositivo così da poter raggrupparli basandosi su dei parametri potenzialmente utili, dato che in un futuro potrebbe essere necessario poter operare su insiemi di dispositivi usando una *wildcard* simile a questa «<manufacturer>/<model>/#» senza, quindi, essere costretti ad utilizzare un numero elevato di topic più specifici.

Successivamente, per alcuni canali di comunicazione, è stato aggiunto l'**identificativo** del dispositivo; questo perché per le categorie di messaggi **management** e **control** è necessario, per il backend di KanbanBOX, poter leggere la risposta ad un comando inviato ad un dispositivo specifico per poter dare un feedback corretto all'utente, quindi la soluzione più conveniente è stata quella di usare un topic dedicato per ogni dispositivo, così da poter rimanere in ascolto sul topic di risposta usato dal dispositivo.

Rimane comunque possibile usare una *wildcard* per operare raggruppando i dispositivi per produttore e modello.

Infine, all'ultimo livello della gerarchia si trova la **macro-categoria di messaggi** per cui quel topic viene utilizzato.

Di seguito viene riportata la struttura dei topic attualmente in uso, associata alla categoria del topic per cui viene usata:

- **management events** e **tag data events**: <manufacturer>/<model>/events, in questo caso si è deciso di usare un solo topic per entrambe le categorie di messaggi dato che tramite la *IoT Rule* implementata [§5.1.3.] il *clientId* viene incluso nel payload del messaggio prima di essere inserito nella coda SQS; così facendo si evita di dover configurare due *rule* diverse per instradare le due categorie di messaggi verso la stessa coda;
- **management**: come già accennato, è necessario usare dei topic dedicati per ogni dispositivo dato che è necessario leggere le risposte ai comandi inviati
 - ▶ <manufacturer>/<model>/<clientId>/management_commands
 - ▶ <manufacturer>/<model>/<clientId>/management_responses
- **control**: anche in questo caso sono necessari dei topic dedicati
 - ▶ <manufacturer>/<model>/<clientId>/control_commands
 - ▶ <manufacturer>/<model>/<clientId>/control_responses

5.1.3. AWS IoT Core

Come già accennato in precedenza, AWS IoT Core è un servizio di Amazon Web Services che integra un broker MQTT e una serie di funzionalità a supporto della gestione di dispositivi IoT e della raccolta, elaborazione o distribuzione dei dati da essi generati.

In AWS IoT ogni reader RFID è rappresentato da una **Thing**, ovvero un'entità che rappresenta un dispositivo fisico; ogni *Thing* registrata in un account AWS è identificata da un nome univoco a livello di regione AWS. Nell'infrastruttura di KanbanBOX avremo una *Thing* che permette al backend di connettersi ad AWS IoT

come se fosse un client MQTT in modo che questo possa inviare e ricevere i messaggi usando il broker integrato, e una *Thing* per ogni reader RFID configurato. Per ogni *Thing* è possibile definire degli attributi, un gruppo di appartenenza e un tipo. In questo caso si è deciso di definire gli attributi **manufacturer** e **model** che rappresentano rispettivamente il produttore e il modello del dispositivo rappresentato dalla *Thing*; attualmente questi attributi sono utilizzati per definire la **struttura dei topic** [§5.1.2.] ma, in futuro, potrebbero essere utili per operare su insiemi di dispositivi raggruppandoli per produttore o modello.

Al momento della creazione di una *Thing* è necessario anche **associare un certificato**, che viene utilizzato per l'autenticazione del dispositivo rappresentato dalla *Thing* quando questo si connette ad AWS IoT Core.

AWS IoT permette di scegliere se generare un certificato X.509 direttamente da AWS o se utilizzare un certificato generato esternamente; dato che si è deciso di gestire, e quindi anche di creare, le *Thing* associate ai reader registrati su KanbanBOX tramite l'SDK di AWS IoT per PHP, è risultato molto più pratico utilizzare certificati generati da AWS ottenibili e assegnabili alle *Thing* tramite i metodi implementati dall'SDK stesso.

Ad ogni certificato viene poi associata una **policy**, ovvero un file JSON che definisce i permessi di accesso alle risorse AWS per le *Thing* (nel nostro caso ogni certificato sarà dedicato ad una sola *Thing*) a cui è associato il certificato.

Per l'infrastruttura di KanbanBOX sono state definite due policy:

- **Administration**: questa policy è associata alla *Thing* che rappresenta il **backend di KanbanBOX**, e permette a questa di connettersi ad AWS IoT Core, pubblicare e ricevere messaggi su tutti i topic, creare e gestire le *Thing* associate ai reader RFID, e gestire i certificati e le policy

```
1  {
2    "Version": "2012-10-17",
3    "Statement":
4    [
5      {
6        "Effect": "Allow",
7        "Action": [
8          "iot:Publish",
9          "iot:Receive",
10         "iot:Republish",
11         "iot:Subscribe",
12         "iot:Connect",
13         "iot:GetRetainedMessage",
14         "iot:ListRetainedMessages",
15         "iot:RetainPublish",
16         "iot:CreateKeysAndCertificate",
17         "iot>DeleteCertificate",
18         "iot:AttachPolicy",
19         "iot:DetachPolicy"
20       ],
21       "Resource": "*"
22     }
```



```

23   ]
24 }

```

- ***Connect_Publish_Subscribe_Receive_ByThingName***: questa policy è associata ad ogni *Thing* che rappresenta un reader RFID; in questo caso vengono usate le **variabili** messe a disposizione da **AWS IoT Core nelle policy** per ottenere, dinamicamente, parametri relativi alla connessione MQTT in corso.

Di seguito è stata inserita una parte della policy dove vengono mostrati solamente i permessi di connessione e pubblicazione (dato che gli altri permessi presenti sono analoghi a quello di pubblicazione); si può notare come per la connessione venga usata la variabile ***\${iot:Connection.Thing.ThingName}*** per obbligare il reader a connettersi solamente usando il *clientId* (che è equivalente al *ThingName* su AWS) associato al certificato che sta utilizzando; infatti AWS IoT, una volta ricevuta la richiesta di connessione, verificherà che il certificato usato dal reader sia associato alla *Thing* corrispondente al *clientId* usato in questa fase dal reader.

Il permesso di pubblicazione sfrutta anche la variabile ***\${iot:Connection.Thing.Attributes[...]}*** per recuperare gli attributi della *Thing*, e imporre al reader di pubblicare solo sui topic a lui dedicati.

```

1  {
2    "Version": "2012-10-17",
3    "Statement":
4    [
5      {
6        "Effect": "Allow",
7        "Action": "iot:Connect",
8        "Resource": "<endpoint-arn>:client/${iot:Connection.Thing.ThingName}"
9      },
10     {
11       "Effect": "Allow",
12       "Action": "iot:Publish",
13       "Resource": [
14         "<endpoint-arn>:topic/${iot:Connection.Thing.Attributes[manufacturer]}/${iot:Connection.Thing.Attributes[model]}/${iot:Connection.Thing.ThingName}/*",
15         "<endpoint-arn>:topic/${iot:Connection.Thing.Attributes[manufacturer]}/${iot:Connection.Thing.Attributes[model]}/events"
16       ]
17     },
18     // altri permessi ...
19   ]
20 }

```

L'*endpoint-arn* è un *placeholder* che va sostituito con l'ARN dell'endpoint di AWS IoT Core utilizzato per la connessione al broker MQTT.

L'ultimo elemento necessario nell'infrastruttura progettata sono le **IoT Rule**, ovvero uno strumento che permette di **instradare i messaggi** ricevuti su determinati topic nel broker MQTT di AWS IoT Core verso altre risorse AWS.

In questo caso è bastato configurare una sola IoT Rule che instradasse i messaggi

ricevuti sul topic dedicato ai tag RFID e agli *heartbeat* (topic che è lo stesso per tutti i reader) verso una coda SQS; così facendo abbiamo potuto implementare un worker nel backend di KanbanBOX che può estrarre i messaggi dalla coda in modalità *pull* e processarli.

Le *IoT Rule* vengono configurate tramite una query in linguaggio SQL [9], arricchito con delle funzioni specifiche per AWS IoT, che permette di filtrare i messaggi in ingresso e di estrarre i dati che ci interessano. Di seguito viene riportata la query utilizzata:

```
1 SELECT *, clientId() AS clientId FROM '#' WHERE topic(3) = 'events' sql
```

Come vediamo dalla clausola FROM, la query recupera i messaggi ricevuti su tutti i topic (**wildcard #**) e usando la clausola WHERE, abbinata alla **funzione topic(n)** che ritorna l'n-esimo livello del topic, filtra i messaggi per ottenere solo quelli ricevuti sui topic che terminano con *events*, ovvero i topic dedicati ai messaggi dei tag RFID e degli *heartbeat*; come anticipato, nel nostro caso il topic è lo stesso per tutti i reader ma si è deciso di usare una regola dinamica in caso di futuri cambiamenti nella struttura dei topic.

Altro aspetto peculiare è l'utilizzo della **funzione clientId()** che, come per la funzione *topic()*, è implementata nativamente da AWS IoT; questa viene usata per integrare il *clientId* nel payload del messaggio, così da poter identificare il reader che lo ha generato e quindi associare i dati del tag o dell'*heartbeat* al reader corretto direttamente dal backend di KanbanBOX, senza dover configurare un topic ed una *IoT Rule* dedicati per ogni reader, per poi utilizzare quest'ultimi come «canali identificativi».

In aggiunta al *clientId*, ovviamente, vengono inclusi anche tutti gli altri dati del messaggio (*payload*, *timestamp*, tipo, ...) usando l'**asterisco (*)**.

5.1.4. AWS SQS

AWS Simple Queue Service (SQS) è un servizio di **message queuing** che consente di stabilire una comunicazione **asincrona** tra componenti di un sistema software, introducendo una coda persistente tra chi produce eventi e chi li elabora. La coda è detta **asincrona** perché il componente che produce l'evento non deve conoscere né contattare direttamente il destinatario; infatti il messaggio viene pubblicato nella coda e l'elaborazione avviene in un secondo momento, quando il «componente consumatore» (nel nostro caso il worker) legge i messaggi disponibili. Nel sistema descritto, SQS viene impiegato come coda di transito per i messaggi ricevuti su AWS IoT Core: una *IoT Rule* [§5.1.3.] instrada i messaggi, in formato **JSON**, pubblicati sul topic `<manufacturer>/<model>/events` verso la coda SQS, mantenendo quindi separati la trasmissione tramite MQTT e il consumo dei messaggi.

L'uso della coda introduce due vantaggi principali:

- **buffering**: se il worker non riesce a processare i messaggi alla stessa velocità con cui vengono prodotti, questi si accumulano nella coda senza andare persi. Questo permette di assorbire picchi temporanei nel traffico mantenendo stabile il resto del sistema.
- **persistenza**: i messaggi vengono memorizzati da SQS per un periodo configurabile (7 giorni in questa implementazione), rendendo possibile la consegna anche in presenza di indisponibilità temporanee del worker.

Dal punto di vista del modello di consegna, SQS permette una modalità di consumo **pull**: il worker, utilizzando l'SDK AWS per PHP [7], interroga periodicamente la coda e recupera i messaggi disponibili. Questo approccio è utile anche per gestire casi in cui un messaggio, una volta letto, non venga effettivamente considerato d'interesse (ad esempio in caso di eccezione per tipi di messaggi non gestiti lato backend); in questi casi il messaggio non viene confermato come consumato (cioè rimosso dalla coda), e può tornare nuovamente disponibile in coda.

Nei casi in cui il messaggio sia malformato o contenga dati incompleti, invece, il worker lo consuma e lo scarta, evitando che possa essere processato più volte.

Come parzialmente anticipato nella descrizione della struttura dei topic [§5.1.2.], per i messaggi di tipo **management** e **control** è sorta la necessità di poter leggere le risposte ai comandi inviati ai reader, e quindi di gestire le risposte in modo sincrono per poter dare un feedback immediato all'utente.

Per questo motivo si è deciso di utilizzare la coda SQS solamente per i messaggi di dati dei tag e heartbeat e di gestire i messaggi di tipo **management** e **control** direttamente tramite MQTT.

Infine, l'accesso alla coda è regolato tramite **IAM**, infatti è stato creato un ruolo IAM con permessi per scrivere e leggere dalla coda SQS configurata. Questo stesso ruolo è stato associato alla *IoT Rule* per permettergli l'inserimento dei messaggi in coda, e viene usato dal worker di KanbanBOX per permettergli di leggere i messaggi dalla coda; in questo modo si garantisce che solo questi due componenti possano interagire con la coda, e quindi si mantiene un livello di sicurezza adeguato.

5.1.5. KanbanBOX

KanbanBOX è il software gestionale, sviluppato dall'omonima azienda ospitante, che ha lo scopo di facilitare il monitoraggio e la gestione dei processi produttivi e logistici attraverso l'uso di **kanban** digitali che vengono associati a specifici tag RFID.

La piattaforma web di KanbanBOX comunica in tre direzioni distinte nel flusso dei dati relativo al dominio dei reader RFID.

La prima riguarda il **polling dei messaggi** dei tag RFID e degli heartbeat, che tramite l'SDK di AWS (usando il protocollo **HTTP** per le chiamate), vengono estratti dalla coda SQS e processati, da un worker dedicato.

Il processamento consiste nel distinguere la tipologia di messaggio ricevuto e nell'estrazione dei dati rilevanti da esso; nel caso dei messaggi dei tag RFID i dati di interesse sono principalmente l'identificativo del tag letto, il timestamp di lettura, il reader che ha generato il messaggio e i dati tecnici di lettura (RSSI_G, antenna, numero di letture, ecc.), questi vengono poi utilizzati per aggiornare lo stato del **kanban** associato a quel tag, se esistente, e per mostrare la lettura del cartellino nella dashboard dei tag letti.

Mentre per i messaggi di heartbeat i dati di interesse sono l'identificativo del reader e il timestamp di ricezione del messaggio, utilizzati per **aggiornare lo stato di connessione** del reader in modo da informare l'utente sull'operatività del reader stesso.

Il secondo flusso di dati riguarda la **configurazione dei reader** tramite l'interfaccia web di KanbanBOX, che tramite il protocollo MQTT invia comandi ai reader per configurare i parametri di connessione e la modalità operativa e riceve l'esito dell'applicazione dei comandi dai reader.

Tutti i dati riguardanti a questo flusso passano per il broker **MQTT** di AWS IoT Core, attraverso i topic descritti nella sezione dedicata alla struttura dei topic [§5.1.2].

L'ultimo flusso è relativo alla comunicazione verso AWS IoT Core tramite l'SDK di AWS per PHP, che viene utilizzato principalmente per la **gestione delle entità di AWS IoT** Core, in particolare per la ricezione del certificato e delle chiavi necessari per la generazione del file PFX.

Capitolo 6.

Codifica

6.1. Design pattern utilizzati

6.1.1. Repository

Il **Repository pattern** è un pattern architetturale che astrae l'accesso ai dati (principalmente contenuti in un DB) e fornisce un'interfaccia che semplifica le operazioni di lettura/scrittura, nascondendo dettagli come query SQL, ORM o filesystem. L'obiettivo principale è **separare** la logica di business dalla logica di persistenza, mantenendo il codice più testabile e manutenibile.

Nel codice mostrato viene usato in più punti:

- **RfidReaderRepository** viene utilizzato per recuperare i dati del reader a partire dal suo id (ad es. in `RfidDownloadCertificate::handle(...)` e in `DeleteRfidReaderRow::execute(...)`). In entrambi i casi la logica non contiene query o accesso diretto al DB ma si limita ad interrogare il repository e ad applicare la logica di business;
- **RfidScanRepository** viene usato in `RfidEventsMessageHandler` per salvare l'evento `RfidScan` dopo la validazione (`store($event)`), mantenendo l'handler focalizzato sul ricezione e validazione dei dati letti.

6.1.2. Dependency Injection

La **Dependency Injection** (DI) è un design pattern in cui un oggetto non crea autonomamente le proprie dipendenze, ma le riceve da un *container*, ovvero una classe dedicata a questo scopo. In questo modo si riduce l'accoppiamento tra componenti e si ottiene un codice più modulare e testabile.

Nel progetto la DI è gestita principalmente tramite il **Container di Symfony**: il suo scopo è definire **come istanziare** tutti gli oggetti dell'applicazione e **quali dipendenze** debbano ricevere. In pratica, il container descrive il grafo delle dipendenze e si occupa di costruire i servizi nel momento in cui servono, rispettando l'ordine corretto e riusando le istanze quando previsto.

In realtà, come già detto, viene utilizzato nella creazione di sostanzialmente tutte le classi che verranno descritte in questo documento ma nel codice mostrato risulta evidente quando si parla della classe **Container** dove viene configurato tutto ciò che serve per eseguire il consumo dei messaggi da SQS. In particolare:

- viene costruito il receiver SQS tramite `AwsSqsFactory->buildReceiver(...)`, a cui viene passato il serializer `RfidEventsMessageSerializer` (anch'esso costruito con le sue dipendenze, come `Clock` e `LoggerInterface`);
- viene istanziato `ConsumeMessagesCommand`, iniettandogli oggetti già pronti come `RoutableMessageBus`, il receiver locator, l'`EventDispatcher` e il `Logger`.

6.1.3. Factory

Il **Factory pattern** raccoglie la logica di creazione di oggetti in metodi dedicati, così da:

- evitare costruttori troppo complessi «sparsi» nel codice chiamante;
- centralizzare regole di costruzione e default;
- rendere più semplice cambiare implementazioni o parametri in futuro.

Nel codice si vede in:

- **AwsSqsFactory** è una *factory class* che implementa diversi metodi per creare istanze di oggetti atte ad interagire in diverse modalità (ad es. code standard o FIFO) con le code SQS; in questo progetto è stato usato il metodo `buildReceiver(...)` capace di eseguire del *polling* da una coda SQS standard;
- **UpdateReaderCommand::forConfiguration(...)** e **UpdateReaderCommand::consumeCertificate(...)** sono *factory method*, infatti invece di avere un unico costruttore con molti parametri opzionali, si espone un metodo che indica lo scopo specifico;
- **ReaderReportContainsHeartbeat::raise(...)** e **ReaderReportContainsHeartbeat::from(...)** sono *factory method* per creare l'evento assegnando `raisedAt` tramite `Clock` o ricostruendolo da dati serializzati.

6.1.4. Command

Il **Command pattern** rappresenta un'azione sotto forma oggetto: un comando contiene i dati necessari per eseguire un'operazione, e viene processato da un componente dedicato (**CommandBus**). Questo favorisce separazione tra chi **richiede** l'azione e chi la **esegue**, oltre a rendere più modulare l'utilizzo delle operazioni implementate.

Nel codice è usato:

- nella **gestione reader**, operazioni come aggiunta/aggiornamento/rimozione vengono delegate a comandi (`AddReaderCommand`, `UpdateReaderCommand`, `RemoveReaderCommand`);
- nel **flusso dei messaggi**, l'handler degli eventi heartbeat traduce l'evento in un comando `UpdateLastHeartbeatOfTheReaderCommand`, che poi viene eseguito via `CommandBus` per aggiornare il DB.
- lato **worker**, `ConsumeMessagesCommand` incapsula il consumo dei messaggi dalla coda e la loro elaborazione, delegando l'instradamento al bus.

6.1.5. Handler

Il termine **Handler** viene usato per indicare un componente che gestisce un input e compie delle operazioni di conseguenza; può trattarsi di una richiesta HTTP (**RequestHandler**), di un messaggio asincrono (**MessageHandler**), o dell'esecuzione di un comando (non mostrato esplicitamente ma presente in alcune implementazioni nel **CommandBus**).

Nel progetto abbiamo:

- **RfidDownloadCertificate implements RequestHandlerInterface** gestisce una richiesta HTTP, recupera dati dal repository, interagisce con filesystem e comandi, e infine costruisce una `Response` con gli header corretti per il download;

- **RfidEventsMessageHandler implements MessageHandler** che gestisce i messaggi decodificati dalla coda SQS; in base al tipo ricevuto (ReportEventsMessage o RfidScan) decide come gestirlo e che cosa restituire.

6.2. Gestione dei reader RFID

6.2.1. AwsIotClientImplementation

AwsIotClientImplementation è un componente che implementa l'interfaccia **AwsIotClient** e fa da *wrapper/adapter* verso l'SDK di AWS IoT Core per PHP. L'obiettivo è fornire al resto dell'applicazione un'API adatta all'uso nel contesto di KanbanBOX.

I metodi esposti corrispondono alle operazioni necessarie per gestire le entità AWS IoT legate a un reader RFID:

- **__construct(AwsIotConfig \$iotConfig)**: inizializza il client AWS con region e credenziali lette dai metodi statici di configurazione (forniti da delle classi dedicate e separate per ambiente di esecuzione);
- **createThing(...)**: crea una *Thing* (una per reader) valorizzando gli attributi utili (es. manufacturer/model/version);
- **updateThing(...)**: aggiorna gli attributi di una *Thing* esistente;
- **deleteThing(...)**: elimina la *Thing* associata al reader;
- **createCertificate()**: genera un certificato X.509 e la relativa key pair, restituendo il Result dell'SDK (contenente arn e file pem di certificato e chiavi);
- **deleteCertificate(...)**: disattiva un certificato (operazione richiesta da AWS prima della cancellazione) e lo elimina definitivamente da AWS IoT;
- **attachPrincipalPolicy(...)**: associa una policy IoT a un certificato (principal);
- **detachPrincipalPolicy(...)**: rimuove l'associazione tra policy e certificato;
- **listThingPrincipals(...)**: recupera la lista dei certificati attualmente associati alla *Thing* del reader;
- **attachThingPrincipal(...)**: associa un certificato alla *Thing* del reader (con possibilità di definire se il certificato è esclusiva o non-esclusiva);
- **detachThingPrincipal(...)**: rimuove l'associazione tra certificato e *Thing*.

```

1  <?php
2  final class AwsIotClientImplementation implements AwsIotClient
3  {
4      public function __construct(
5          AwsIotConfig $iotConfig,
6      ) {}
7
8      public function createThing(
9          RfidReaderId $clientId,
10         RfidReaderManufacturer $manufacturer,
11         RfidReaderModel $model,
12     ): void {}
13
14     public function updateThing(
15         RfidReaderId $clientId,
```

PHP

```

16     RfidReaderManufacturer $manufacturer,
17     RfidReaderModel $model,
18 ): void {}
19
20 public function deleteThing(RfidReaderId $clientId): void {}
21
22 public function createCertificate(): Result {}
23
24 public function deleteCertificate(string $certificateId): void {}
25
26 public function attachPrincipalPolicy(string $certificateArn, AwsIotPolicy $policyName): void {}
27
28 public function detachPrincipalPolicy(string $certificateArn, AwsIotPolicy $policyName): void {}
29
30 public function listThingPrincipals(RfidReaderId $clientId): array {}
31
32 public function attachThingPrincipal(
33     RfidReaderId $clientId,
34     string $certificateArn,
35     bool $isExclusive,
36 ): void {}
37
38 public function detachThingPrincipal(RfidReaderId $clientId, string $certificateArn): void {}
39 }

```

6.2.2. MqttClientAws

MqttClientAws è un'implementazione dell'interfaccia **MqttClient** che funge da *wrapper/adaptor* verso la libreria **php-mqtt** ([3]) per fornire un'implementazione più completa e semplice da usare all'interno di KanbanBOX.

Il **costruttore** riceve come parametri due DTO contenenti tutti i parametri di configurazione e connessione necessari per un client MQTT.

I metodi **connect()** e **disconnect()** non sono altro che dei *wrapper* che semplificano la chiamata degli omonimi metodi della libreria **php-mqtt** passando di default i parametri di connessione.

Il metodo **publish(...)** si occupa di pubblicare un messaggio su un topic specificato e di rimanere in ascolto su un topic di risposta per un tempo definito, così da poter restituire l'esito dell'operazione al chiamante; durante questo periodo di ascolto viene eseguita una *callable*, passata come parametro la metodo **publish(...)** della libreria, che itera su ogni messaggio ricevuto ed esegue un *early interrupt* se la risposta è quella attesa. Questo metodo è particolarmente utile per inviare comandi ai reader e ricevere una risposta sincrona.

Per la ricezione delle risposte si sfrutta il metodo **publish(...)** che non fa altro che configurare un *loop* durante il quale eseguire la *callable* fornita e semplificare la chiamata al metodo **subscribe()** della libreria adattandolo al caso d'uso di KanbanBOX.

Nonostante l'implementazione dei due metodi precedenti possa risultare ambigua, in [questa issue](#) aperta nella repository della libreria **php-mqtt** si è cercato di verificare che

non ci fossero metodi migliori per raggiungere il nostro scopo e il proprietario della repository stesso ha suggerito di procedere definendo un *loop*.

Abbiamo poi il metodo **publishCommandInTopic(...)** che è un *helper* che gestisce il publish e il subscribe dei comandi usati per configurare i reader assolvendo l'onere di costruire i topic corretti in base alla struttura definita [§5.1.2.] e al formato del messaggio.

Compie un ruolo simile il metodo **setTagReading(...)** che si occupa di inviare comandi di avvio o interruzione della lettura dei tag ai reader.

In entrambi i metodi viene restituito l'esito e, nel caso fosse presente, il messaggio di errore ritornato dai reader.

```
1  <?php
2  class MqttClientAws implements MqttClient
3  {
4      private PhpMqttClient $mqttClient;
5
6      public function __construct(
7          private readonly ConnectionSettings $connectionSettings,
8          MqttConfig $mqttConfig,
9      ){
10         $this->mqttClient = new PhpMqttClient(
11             $mqttConfig->brokerDomain,
12             $mqttConfig->port,
13             $mqttConfig->kanbanBoxThingUuid,
14             PhpMqttClient::MQTT_3_1_1,
15         );
16     }
17
18     public function connect(): void
19     {
20         $this->mqttClient->connect($this->connectionSettings);
21     }
22
23     public function publish(string $topic, string $responseTopic, ZebraCommand $command): bool|string
24     {
25         $this->mqttClient->publish($topic, $command->toString(), PhpMqttClient::QOS_AT_LEAST_ONCE);
26
27         $sentCommandId = $command->commandId;
28
29         $response = null;
30         $this->subscribe(
31             $responseTopic,
32             function (string $topic, string $message) use (&$response, $sentCommandId): void {
33                 $response = json_decode($message, true, flags: JSON_THROW_ON_ERROR);
34
35                 if (($response['command_id'] ?? null) !== $sentCommandId) {
36                     $response = null;
37                 }
38             }
39         );
40     }
41 }
```

```

38         return;
39     }
40
41     $this->mqttClient->interrupt();
42     },
43     15,
44 );
45
46 if ($response === null) {
47     return 'No response received within the timeout period.';
48 }
49 $decodedResponse = $response;
50 return ($decodedResponse['response'] ?? null) === 'success' ? true : ($decodedResponse['payload']
51     ['message'] ?? 'Unknown error');
52 }
53
54 public function subscribe(string $topic, callable $callback, float|null $timeOutSeconds = null): void
55 {
56     if ($timeOutSeconds !== null) {
57         $this->mqttClient->registerLoopEventHandler(static function (PhpMqttClient $client, float
58             $elapsedTimeSeconds) use ($timeOutSeconds): void {
59             if ($elapsedTimeSeconds < $timeOutSeconds) {
60                 return;
61             }
62             $client->interrupt();
63         });
64     }
65     $this->mqttClient->subscribe(
66         $topic,
67         $callback,
68         PhpMqttClient::QOS_AT_LEAST_ONCE,
69     );
70
71     $this->mqttClient->loop(false);
72 }
73
74 public function publishCommandInTopic(
75     TopicLeaf $publishTopicLeaf,
76     TopicLeaf $responseTopicLeaf,
77     ZebraCommand $command,
78     RfidReaderManufacturer $readerManufacturer,
79     RfidReaderModel $readerModel,
80     RfidReaderId $readerId,
81     RfidImplementationVersion $rfidImplementationVersion = RfidImplementationVersion::V1,
82 ): bool|string {
83     $publishTopic = sprintf(

```

```

84     '%s/%s/%s/%s/%s',
85     $rfidImplementationVersion->value,
86     $readerManufacturer->getValue(),
87     $readerModel->getValue(),
88     $readerId,
89     $publishTopicLeaf->value,
90 );
91
92     $subscribeTopic = sprintf(
93     '%s/%s/%s/%s/%s',
94     $rfidImplementationVersion->value,
95     $readerManufacturer->getValue(),
96     $readerModel->getValue(),
97     $readerId,
98     $responseTopicLeaf->value,
99 );
100
101     return $this->publish($publishTopic, $subscribeTopic, $command);
102 }
103
104 public function disconnect(): void
105 {
106     $this->mqttClient->disconnect();
107 }
108
109 public function setTagReading(
110     RfidReader $reader,
111     ZebraCommands $command,
112     DateTimeImmutable $now,
113     RfidImplementationVersion $rfidImplementationVersion = RfidImplementationVersion::V1,
114 ): bool|string {
115     return $this->publishCommandInTopic(
116         TopicLeaf::Controls,
117         TopicLeaf::ControlsResponses,
118         new ZebraCommand(
119             $command,
120             $now,
121             '{}',
122         ),
123         RfidReaderManufacturer::from($reader->getManufacturer()->value),
124         RfidReaderModel::from($reader->getModel()->value),
125         $reader->getId(),
126         $rfidImplementationVersion,
127     );
128 }
129 }

```

6.2.3. SetupNewReaderOnAwsIot

Il servizio **SetupNewReaderOnAwsIot** si occupa di eseguire tutte le operazioni necessarie per creare e configurare una nuova *Thing* su AWS IoT Core associata al reader RFID che si sta aggiungendo su KanbanBOX. Inoltre si occupa di gestire il *rollback* nel caso in cui una qualsiasi delle operazioni fallisca, così da mantenere la coerenza tra AWS IoT Core e il database di KanbanBOX evitando di lasciare entità orfane o non completamente configurate su AWS; in questi casi viene ritornato un oggetto che implementa **ErrorCreatingEntity** che permette di gestire in modo più granulare i feedback di errore da dare all'utente. Se la creazione va a buon fine ritorna un DTO che contiene tutti i dati necessari per la generazione del certificato PFX.

Per assolvere al suo scopo, **SetupNewReaderOnAwsIot** utilizza i metodi di **AwsIotClientImplementation**.

```
1  <?php
2  final readonly class SetupNewReaderOnAwsIot implements SetupNewReaderOnExternalPlatform
3  {
4      public function __construct(
5          private AwsIotClient $awsIotClient,
6          private GetConfig $getConfig,
7          private LoggerInterface $logger,
8      ){
9      }
10
11     public function __invoke(
12         RfidReaderId $readerId,
13         RfidReaderManufacturer $manufacturer,
14         RfidReaderModel $model,
15     ): ErrorCreatingEntity|IotCertificate {
16         try {
17             $this->awsIotClient->createThing(
18                 $readerId,
19                 $manufacturer,
20                 $model,
21             );
22         } catch (Throwable $e) {
23             $this->logError($e, $readerId);
24
25             return new ErrorCreatingThing();
26         }
27
28         try {
29             $awsCertificate = $this->awsIotClient->createCertificate();
30             $rootCaCertificateContent = file_get_contents($this->getConfig->getAmazonRootCaCertificateFilePath());
31             $certificate = new IotCertificate(
32                 $awsCertificate['certificateId'],
33                 $awsCertificate['certificateArn'],
34                 $awsCertificate['certificatePem'],
```

```

35     $awsCertificate['keyPair']['PrivateKey'],
36     $this->getConfig->getPfxCertificateEncryptionKey(),
37     $rootCaCertificateContent,
38     );
39 } catch (Throwable $e) {
40     $this->logError($e, $readerId);
41
42     $this->rollbackCreatedThing($readerId);
43
44     return new ErrorCreatingCertificate();
45 }
46
47 try {
48     $this->awsIotClient->attachPrincipalPolicy($certificate->arn, AwsIotPolicy::CLIENT);
49 } catch (Throwable $e) {
50     $this->logError($e, $readerId);
51
52     $this->rollbackCreatedThing($readerId);
53     $this->rollbackCreatedCertificate($certificate->id);
54
55     return new ErrorAttachingPrincipalPolicy();
56 }
57
58 try {
59     $this->awsIotClient->attachThingPrincipal($readerId, $certificate->arn, true);
60 } catch (Throwable $e) {
61     $this->logError($e, $readerId);
62
63     $this->rollbackCreatedThing($readerId);
64     $this->rollbackCreatedCertificate($certificate->id);
65     $this->rollbackAttachedPolicy($certificate->arn, AwsIotPolicy::CLIENT);
66
67     return new ErrorAttachingThingPrincipal();
68 }
69
70 return $certificate;
71 }
72
73 private function rollbackCreatedThing(RfidReaderId $readerId): void
74 {
75     $this->awsIotClient->deleteThing($readerId);
76 }
77
78 private function rollbackCreatedCertificate(string $certificateId): void
79 {
80     $this->awsIotClient->deleteCertificate($certificateId);
81 }
82

```

```

83     private function rollbackAttachedPolicy(string $certificateArn, AwsIotPolicy $policyName): void
84     {
85         $this->awsIotClient->detachPrincipalPolicy($certificateArn, $policyName);
86     }
87
88     private function logError(Throwable $error, RfidReaderId $readerId): void
89     {
90         $this->logger->error(
91             sprintf(
92                 'Error setting up new reader %s on AWS IoT -> %s',
93                 $readerId->id->toString(),
94                 $error->getMessage(),
95             ),
96         );
97     }
98 }

```

6.2.4. UpdateReaderOnAwsIot

UpdateReaderOnAwsIot è un servizio che si occupa di aggiornare le informazioni di una *Thing* esistente su AWS IoT Core in caso di modifica dei dati del reader (ad esempio modello o produttore) su KanbanBOX.

Anche in questo caso per compiere l'aggiornamento vengono usati i metodi della classe **AwsIotClientImplementation**.

```

1  <?php
2  readonly class UpdateReaderOnAwsIot implements UpdateReaderOnExternalPlatform
3  {
4      public function __construct(
5          private AwsIotClient $awsIotClient,
6          private LoggerInterface $logger,
7      ) {
8      }
9
10     public function __invoke(
11         RfidReaderId $rfidReaderId,
12         RfidReaderManufacturer $manufacturer,
13         RfidReaderModel $model,
14     ): true|ErrorUpdatingThing {
15         try {
16             $iotClient = $this->awsIotClient;
17
18             $iotClient->updateThing(
19                 $rfidReaderId,
20                 $manufacturer,
21                 $model,
22             );
23         } catch (Throwable $e) {
24             $this->logger->error(

```

```

25     sprintf(
26         'Error updating thing on AWS IoT for reader %s: %s',
27         $rfidReaderId->id->toString(),
28         $e->getMessage(),
29     ),
30 );
31
32     return new ErrorUpdatingThing();
33 }
34
35     return true;
36 }
37 }

```

6.2.5. PfxHelper

La classe **PfxHelper** ha il solo scopo di adattare, al contesto di KanbanBox, la chiamata al metodo `openssl_pkcs12_export(...)` della libreria `openssl`, per generare un file PFX a partire dai file in formato PEM forniti da AWS IoT.

```

1  <?php
2  class PfxHelper
3  {
4      public function getPfxAsString(IotCertificate $certificate): string
5      {
6          $pfxFile = null;
7
8          openssl_pkcs12_export($certificate->certificatePem, $pfxFile, $certificate->privateKey, $certificate->passphrase, (array) $certificate->extraCertificates);
9
10         return $pfxFile;
11     }
12 }

```

6.2.6. Reader

In questo contesto la classe `Reader` può essere vista come un *controller*, infatti permette all'interfaccia utente, ovvero la **tabella** per la gestione dei **reader RFID**, di interagire con la **logica di business** che si occupa di riflettere le modifiche a DB e sui servizi esterni (ad esempio quelli di AWS). Per generare e rendere funzionale questa tabella vengono usate diverse classi che possono essere *middleware* o veri e propri servizi di supporto; in questa sezione verrà descritta solo la classe `Reader` e alcune delle classi ausiliare usate da quest'ultima, dato che sono quelle più rilevanti e su cui ho lavorato durante questo progetto.

Di seguito vengono mostrate alcune immagini raffiguranti la tabella e le relative funzionalità, assieme alle rispettive implementazioni e descrizioni degli aspetti più tecnici. Data la dimensione della classe `Reader` e la quantità di funzionalità che questa implementa, è stato deciso di riportare solo le porzioni di codice più rilevanti.

Configured RFID Readers									
Reader Name	Manufacturer	Model	MAC Address	Reader status	Reader Connection	Last edit	Last edit user	Creation time	Operations
									4 rows
reader MQTT 1	Zebra	FX9600		Active	Connected	12/04/26	Developer	23/03/26	     
kbb				Active	No	26/03/26	Developer	23/03/26	 
reader HTTP 1	Zebra	FX7500		Active	No	12/04/26	Developer	19/02/26	   
reader MQTT 2	Zebra	FX7500		Active	No	26/03/26	Developer	23/03/26	    

Figura 2: Tabella di gestione dei reader RFID

Nello specifico, in questo contesto, ho lavorato sull'aggiunta del reader che viene definita all'interno di KanbanBOX come **table operation**, in quanto è un'operazione che interagisce con l'entità tabella a differenza delle **row operation** che, invece, operano su una riga della tabella, ovvero i singoli reader.

Riguardo le **row operation**, che si possono vedere nella colonna «Operations», ho lavorato sulle funzionalità di modifica (seconda icona della prima riga), configurazione (terza icona della prima riga) e cancellazione (quarta icona della prima riga) dei reader, e, infine, alla gestione del download dei certificati dei reader (quinta icona della prima riga). Come si può notare, in alcune righe non sono presenti tutte le **row operation**, questo perché:

- per i reader configurati per usare il protocollo HTTP viene inibita la configurazione, in quanto non è previsto l'invio di comandi tramite l'interfaccia di KanbanBOX;
- per i reader HTTP e per quelli di cui è stato scaricato il certificato viene rimossa l'icona di download del certificato (icona della chiave), in quanto non è previsto poter scaricare più volte il certificato di un reader; oltre a questo viene rimosso anche il nome del file contenente il certificato dal DB e viene cancellato il certificato il file stesso dal *filesystem*;
- il reader «kbb» rappresenta il backend di KanbanBOX perché, come spiegato anche nella sezione §5.1.3., è necessario che il backend abbia una *Thing* associata su AWS IoT Core per poter connettersi al broker MQTT e gestire i reader; questo «reader», quindi, non è configurabile in alcun modo, e non è necessario scaricare il suo certificato dato che sarà già presente di default nei secrets di KanbanBOX.

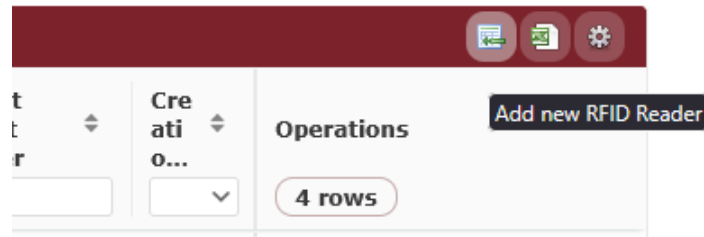


Figura 3: Bottone di aggiunta del reader RFID

 The image displays the 'Add new RFID Reader' form. At the top, there is a dark red header bar with the 'kanbanBOX' logo and navigation links: Home, Master data, Kanbans, Print, Kanban boards, Heijunka, Documents, Set, and Report. A 'Settings' link with a gear icon is also present. Below the header, a dropdown arrow is visible. The form itself is titled 'Add new RFID Reader' and contains several input fields: 'Protocol' (a dropdown menu with 'Http' selected), 'Reader Name' (a text input field), 'Manufacturer' (a dropdown menu with 'Zebra' selected), 'Model' (a dropdown menu with 'FX7500' selected), 'MAC Address' (a text input field), and 'Active Reader' (a dropdown menu with 'Yes' selected). At the bottom of the form, there are two buttons: 'Save' (with a green checkmark icon) and 'Cancel' (in blue text). A circular help icon with a question mark is located in the bottom right corner of the page.

Figura 4: Form di aggiunta e modifica dei reader RFID

La funzione di **aggiunta del reader**, raggiungibile tramite la *table operation* mostrata nella Figura 3, visualizza a schermo il form visibile nella Figura 4; se vengono inseriti i campi obbligatori (con il bordo rosso) e si preme il pulsante «Save», dopo aver superato la validazione del form, si procede con le seguenti *task*:

- se il reader utilizza il protocollo MQTT, viene eseguita la creazione e configurazione delle entità di AWS IoT Core associate al reader tramite il metodo `create_entity_on_aws(...)`; quest'ultimo si occupa di invocare il servizio **SetupNewReaderOnAwsIot** [§6.2.3.] e, se il «setup» va a buon fine, genera il certificato in formato PFX e lo restituisce;
- se il certificato è stato generato (la funzione precedente ha ritornato un `IotCertificate`), o se il reader utilizza il protocollo HTTP, si procede con la memorizzazione del reader a DB tramite il metodo `store_reader(...)`, che si occupa di invocare il comando **AddReaderCommand** [§6.1.4.] per salvare il reader a DB, e di dare un feedback positivo all'utente nel caso fosse andato tutto a buon fine;

- a. se c'è stato qualche errore durante la creazione delle entità su AWS IoT Core, tramite il **RfidReaderCreationErrorPresenter** viene mostrato un messaggio di errore all'utente con le informazioni relative al problema riscontrato;
- b. altrimenti si ritorna alla pagina di gestione dei reader.

```

1  <?php
2  public function add_reader(string|null $readerId = null): void
3  {
4      /* ... codice di supporto per visualizzazione del form e validazione dei dati ... */
5      if ($isMqtt) {
6          $awsThingCreationResult = $this->create_entity_on_aws(
7              $readerId,
8              $rfidReaderManufacturer,
9              $rfidReaderModel,
10             $licenseId,
11         );
12
13         if ($awsThingCreationResult instanceof IotCertificate) {
14             $pfxCertificate = $this->pfxHelper->getPfxAsString($awsThingCreationResult);
15             $pfxStream = Utils::streamFor($pfxCertificate);
16
17             $pfxFileName = $awsThingCreationResult->id . '.pfx';
18             $this->defaultUpload->fileStream(
19                 $pfxFileName,
20                 $pfxStream,
21                 $licenseId,
22                 FileUploadContext::AWS_IOT_CERTIFICATE
23             );
24
25             $this->store_reader(
26                 $readerId,
27                 $pfxFileName,
28                 $isMqtt
29             );
30
31             new ResponseEmitter()->emit(
32                 $this->dic->get(Redirect::class)->toUri(
33                     'rfid/reader/readers',
34                     RedirectMethod::LOCATION,
35                     303
36                 )
37             );
38
39             return;
40         }
41
42         $this->rfidReaderCreationErrorPresenter->__invoke($awsThingCreationResult);
43     } else {

```

```

44     $this->store_reader(
45         $readerId,
46         null,
47         $isMqtt
48     );
49
50     new ResponseEmitter()->emit(
51         $this->dic->get(Redirect::class)->toUri(
52             'rfid/reader/readers',
53             RedirectMethod::LOCATION,
54             303
55         )
56     );
57
58     return;
59 }
60 }

```

La funzione di **aggiornamento del reader** usa un form pressoché identico a quello dell'aggiunta del reader, se non per il campo «Protocol» che nell'aggiornamento non è modificabile e per i campi che sono tutti facoltativi; inoltre segue lo stesso paradigma dell'aggiunta, con qualche differenza nei servizi chiamati, infatti:

- se il reader utilizza il protocollo MQTT, vengono modificati gli attributi della *Thing* su AWS IoT Core tramite il servizio **UpdateReaderOnAwsIot** [§6.2.4.]; ovviamente nessuna operazione viene svolta sul certificato che rimane tale e quale e non viene rigenerato;
- viene invocato il comando **UpdateReaderCommand** [§6.1.4.] per aggiornare il reader a DB;
- se il reader usa il protocollo MQTT, viene inviato un comando MQTT al reader che avvia la lettura dei tag nel caso in cui il campo «Active Reader» venga impostato a «Yes» oppure ferma la lettura se viene impostato a «No»;
- **a.** se c'è stato qualche errore durante l'aggiornamento delle entità su AWS IoT Core o durante l'invio del comando viene visualizzato un messaggio di errore;
- **b.** altrimenti si ritorna alla pagina di gestione dei reader.

Configure RFID Reader Id = 04c7e367-7eaf-4cfa-9081-e412e8af2fd8

connection JSON configuration

```

{
  "READER-GATEWAY": {
    "endpointConfig": {
      "data": {
        "event": {

```

To choose in which environment to send messages use "endpointConfig/data/event/connections/publishTopic": "production/events" or "testing/events"

operating mode JSON configuration

```

{
  "type": "CUSTOM",
  "antennas": [
    1,
    2,
    3,
    4
  ],

```

Save Cancel

KanbanBOX is powered by KanbanBOX Srl

Figura 5: Form di modifica dei reader RFID

Edit

▼ Rfid Reader Connection Configuration properties

▼ READER-GATEWAY properties

managementEventConfig properties

endpointConfig properties

data properties

event properties

connections

item 1 properties

type

mqtt-AWS

options properties

Cancel Confirm

KanbanBOX is powered by KanbanBOX Srl

Figura 6: JSON schema per la configurazione dei reader RFID

Come anticipato la funzionalità di **configurazione di un reader** è raggiungibile solamente se il reader in questione utilizza il protocollo MQTT, questo perché è necessario poter inviare dei comandi MQTT al reader per configurarlo, e questa

funzionalità non è prevista per i reader HTTP.

Le configurazioni sono inseribili manualmente incollandole in formato JSON oppure generando il JSON tramite l'interfaccia basata su JSON schema mostrata in Figura 6; in entrambi i casi, una volta premuto il pulsante «Save», le configurazioni vengono validate tramite lo schema e successivamente si procede con le seguenti *task*:

- si esegue la connessione al broker MQTT;
- si controlla che almeno una delle due configurazioni sia definita
 - a. se sì, si procede con l'invio dei comandi per cui la configurazione è definita usando il metodo `MqttClient::publishCommandInTopic(...)`;
 - b. altrimenti si ignorano tutti i passaggi successivi e viene eseguito il *redirect* alla tabella dei reader.
- si esegue la disconnessione dal broker MQTT;
- se almeno una delle due configurazioni è andata a buon fine viene eseguito il salvataggio a DB delle configurazioni inviate per cui si è ricevuto esito positivo;
- viene dato un feedback relativo all'esito dell'applicazione delle configurazioni e viene eseguito il *redirect* alla tabella dei reader;

```
1  <?php PHP
2  public function configure_reader(string $readerId): void
3  {
4      /* ... codice di supporto per visualizzazione del form e validazione dei dati ... */
5      $this->mqttClient->connect();
6
7      if (isset($formData['configuration']) && $formData['configuration'] !== '') {
8          $configuration = $formData['configuration'];
9          $configResult = $this->mqttClient->publishCommandInTopic(
10             TopicLeaf::Management,
11             TopicLeaf::ManagementResponses,
12             new ZebraCommand(
13                 ZebraCommands::SET_CONFIG,
14                 $this->clock->now(),
15                 $configuration,
16             ),
17             $readerId,
18             $currentLicense,
19             $this->rfidSignature,
20         );
21     }
22
23     if (isset($formData['operatingModeConf']) && $formData['operatingModeConf'] !== '') {
24         $operatingModeConf = $formData['operatingModeConf'];
25         $operatingModeResult = $this->mqttClient->publishCommandInTopic(
26             TopicLeaf::Controls,
27             TopicLeaf::ControlsResponses,
28             new ZebraCommand(
29                 ZebraCommands::SET_OPERATING_MODE,
30                 $this->clock->now(),
31                 $operatingModeConf,
```

```

32     ),
33     $readerId,
34     $currentLicense,
35     $this->rfidSignature,
36     );
37 }
38 $this->mqttClient->disconnect();
39
40 if ($configResult === true || $operatingModeResult === true) {
41     $command = UpdateReaderCommand::forConfiguration(
42         $readerId,
43         $currentUser,
44         $currentLicense,
45         $configResult === true && $configuration !== "" ? $configuration : null,
46         $operatingModeResult === true && $operatingModeConf !== "" ? $operatingModeConf : null
47     );
48
49     $this->commandBus->execute($command);
50
51     if ($configResult === true) {$this->globalMessage->insert(/* ... configurazione del messaggio di
    successo ... */);}
52     if ($operatingModeResult === true) {$this->globalMessage->insert(/* ... configurazione del messaggio di
    successo ... */);}
53 }
54 if ($configResult === false) {$this->globalMessage->insert(/* ... configurazione del messaggio di errore ...
    */);}
55 if ($operatingModeResult === false) {$this->globalMessage->insert(/* ... configurazione del messaggio di
    errore ... */);}
56 new ResponseEmitter()->emit(
57     $this->dic->get(Redirect::class)->toUri('rfid/reader/readers', RedirectMethod::LOCATION, 303)
58 );
59 return;
60 }

```

6.2.7. DownloadReaderCertificateRow e RfidDownloadCertificate

Configured RFID Readers									
Reader Name	Manufacturer	Model	MAC Address	Reader status	Reader Connection	Last edit	Last edit user	Created on	Operations
reader MQTT 1	Zebra	FX9600		Active	Connected	12/04/26	Developer	23/03/26	4 rows
kbb				Active	No	26/04/26			This operation can be performed only once. Do you really want to proceed?

Figura 7: Row operation di download del certificato

Il download dei certificati è implementato in modo diverso dalle altre row operation poiché, invece di visualizzare un form di cui eseguire il submit, c'è bisogno di

mostrare un pop-up di conferma, e, in questo caso specifico, dopo la conferma è necessario aprire una *blank page* per avviare il download del file. Per questo nella classe di costruzione della tabella dei reader, il bottone della *row operation* di download dei certificati viene associato ad un oggetto di tipo `DownloadReaderCertificateRow`, classe che estende `ExecuteOnRowWithConfirmation` e che, appunto, permette di eseguire un'operazione su una riga della tabella dopo aver chiesto una conferma all'utente.

Tra i **parametri** del costruttore (e del metodo `create` che è analogo al costruttore ma, per definizione delle classi padre astratte, è statico ed è quello che viene effettivamente usato per costruire l'oggetto) ci sono alcune caratteristiche della *row operation* (`id`, `title`, `icon`) e le sue dipendenze, le più rilevanti sono `TwigEnvironment` che è necessaria per poter creare l'elemento grafico usando i template di Twig (viene usato dai costruttori delle classi padre) e `InternalUrlBuilder` che è necessario per costruire il link a cui fare *redirect* dopo la conferma, ovvero il link che permette di scaricare il certificato del reader.

Infatti nel metodo `execute(...)`, che viene invocato quando l'utente conferma l'operazione, viene costruito un oggetto `RedirectToLink` che, una volta restituito, permette di eseguire un *redirect* verso il link di download del certificato del reader; quest'ultimo viene costruito usando l'`InternalUrlBuilder` e punta ad un endpoint del backend di KanbanBOX associato alla classe `RfidDownloadCertificate`, descritta più sotto, che si occupa di restituire il file del certificato da scaricare.

```

1  <?php
2  final class DownloadReaderCertificateRow extends ExecuteOnRowWithConfirmation
3  {
4      protected function __construct(
5          string $id,
6          string $title,
7          string $icon,
8          private readonly Language $language,
9          private readonly GetTranslation $getTranslation,
10         TwigEnvironment $twigEnvironment,
11         private InternalUrlBuilder $urlBuilder,
12     ){
13         parent::__construct($id, $title, $icon, $language, $twigEnvironment, $getTranslation);
14     }
15
16     public static function create(/* analogo al costruttore */): self
17     {
18         return new self(/* ... */);
19     }
20
21     protected function execute(RowId $rowId, Table $table): OperationResponse
22     {
23         return new RedirectToLink(
24             $this->urlBuilder->build('rfid/download_certificate/' . $rowId->id),
25             Target::NewTab,
26             reloadRow: true,

```

```

27     );
28 }
29
30 protected function getAskConfirmationConfiguration(RowId $rowId): AskConfirmationConfiguration
31 {
32     return new AskConfirmationConfiguration(($this->getTranslation)
33         ('rfid_download_certificate_confirmation', $this->language));
34 }

```

Come anticipato, l'endpoint a cui viene fatto *redirect* dopo la conferma dell'operazione di download del certificato punta alla classe `RfidDownloadCertificate`, che è un ***request handler*** e si occupa di gestire il certificato al momento del download, sia dal punto di vista del filesystem che a DB.

Infatti il **costruttore** si occupa solo di ricevere le dipendenze utilizzate dal metodo **handle(...)** che opera come segue:

- recupera le informazioni del reader da DB usando l'identificativo fornito dalla richiesta ricevuta dall'endpoint, controllando che il reader sia effettivamente controllato dalla licenza (identificativo del client) attualmente in uso;
- estrae il nome del file del certificato associato al reader, se è presente, e lo legge dal filesystem; se il nome del certificato non è presente, significa che il certificato è già stato scaricato in precedenza, quindi viene lanciata un'eccezione;
- cancella il file del certificato dal filesystem e rimuove il nome del certificato a DB, in modo da non permettere ulteriori download dello stesso certificato;
- invia un comando per aggiornare il reader a DB, in modo da riflettere il download del certificato e inibire ulteriori download;
- restituisce una risposta con il file del certificato da scaricare, con gli header HTTP corretti per permettere il download del file.

```

1  <?php
2  final readonly class RfidDownloadCertificate implements RequestHandlerInterface
3  {
4      public function __construct(
5          private GetFromSession $getFromSession,
6          private RfidReaderRepository $rfidReaderRepository,
7          private UserFilesFilesystem $userFilesFilesystem,
8          private Upload $upload,
9          private CommandBus $commandBus,
10     ) {}
11
12     public function handle(ServerRequestInterface $request): ResponseInterface
13     {
14         $arguments = $request->getAttribute(UriArguments::class);
15
16         $readerIdArgument = $arguments->get('readerId');
17         $readerId = RfidReaderId::fromString($readerIdArgument);
18         $reader = $this->rfidReaderRepository->getById($readerId);
19

```



```

20     if (! $reader->getLicense()->getId()->>equals($this->getFromSession->licenseId())) {
21         throw new Exception(sprintf('RFID Reader %s does not belong to the license %s.', $readerId->id-
22             >toString(), $this->getFromSession->licenseId()->id));
23     }
24     $certificateName = $reader->getCertificateName();
25     if ($certificateName === null) {
26         throw new Exception(sprintf('RFID Reader %s certificate was already downloaded.', $readerId->id-
27             >toString()));
28     }
29     $readStream = $this->userFilesFilesystem->readStream(
30         $this->upload->getDefaultFilePath($this->getFromSession->licenseId(),
31             FileUploadContext::AWS_IOT_CERTIFICATE, $certificateName),
32     );
33     $this->userFilesFilesystem->delete(
34         $this->upload->getDefaultFilePath($this->getFromSession->licenseId(),
35             FileUploadContext::AWS_IOT_CERTIFICATE, $certificateName),
36     );
37     $this->commandBus->execute(
38         UpdateReaderCommand::consumeCertificate($reader->getId(), $this->getFromSession->userId(),
39             $this->getFromSession->licenseId()),
40     );
41     return new Response(
42         200,
43         [
44             'Content-Type' => 'application/x-pkcs12',
45             'Content-Disposition' => sprintf('attachment; filename="%s"', $certificateName),
46             'Content-Transfer-Encoding' => 'binary',
47         ],
48         $readStream,
49     );

```

6.2.8. DeleteRfidReaderRow

La **cancellazione di un reader** è implementata in modo simile al download dei certificati, infatti anche in questo caso viene mostrato un pop-up di conferma prima di eseguire l'operazione, e viene eseguito un *redirect* alla tabella dei reader dopo la cancellazione.

Analogamente al download dei certificati il costruttore i dati della *row operation* e le dipendenze necessarie per eseguire l'operazione, tra cui **AwsIotClient** che è necessario per eseguire la cancellazione delle entità su AWS IoT Core associate al reader, **RfidReaderRepository** per recuperare le informazioni del reader da DB, e **UserFilesFilesystem** e **Upload** per cancellare il file del certificato associato al reader dal filesystem nel caso in cui fosse presente.

Il metodo `execute(...)` si occupa di eseguire la cancellazione effettiva del reader, che consiste nei seguenti passaggi:

- recuperare le informazioni del reader da DB usando l'identificativo fornito dalla riga su cui è stata eseguita l'operazione, controllando che il reader sia effettivamente controllato dalla licenza (identificativo del client) attualmente in uso;
- se il reader utilizza il protocollo MQTT, esegue la cancellazione delle entità su AWS IoT Core associate al reader tramite il `AwsIotClient`, che si occupa di:
 - recuperare gli arn (identificativi delle risorse su AWS) dei certificati associati al reader tramite la funzione `listThingPrincipals(...)`;
 - per ogni certificato estratto, disassociare il certificato dalla policy associata e dal reader, cancellare il certificato da AWS IoT Core e cancellare il file del certificato dal filesystem;
 - cancellare la *Thing* associata al reader;
- cancellare il file del certificato dal filesystem;
- cancellare il reader a DB tramite il comando **RemoveReaderCommand** [§6.1.4.];
- se c'è stato qualche errore durante la cancellazione delle entità su AWS IoT Core, viene mostrato un messaggio di errore all'utente e viene registrato nei log l'errore riscontrato; altrimenti viene mostrato un messaggio di successo all'utente;
- viene eseguito il *redirect* alla tabella dei reader.

```

1  <?php
2  class DeleteRfidReaderRow extends ExecuteOnRowWithConfirmation
3  {
4      private function __construct(
5          string $id,
6          string $title,
7          string $icon,
8          private readonly LicenseAndUser $licenseAndUser,
9          private readonly Language $language,
10         private readonly GetTranslation $getTranslation,
11         TwigEnvironment $twigEnvironment,
12         private readonly CommandBus $commandBus,
13         private readonly GlobalMessage $globalMessage,
14         private readonly AwsIotClient $awsIotClient,
15         private readonly LoggerInterface $logger,
16         private readonly RfidReaderRepository $rfidReaderRepository,
17         private readonly UserFilesFilesystem $userFilesFilesystem,
18         private readonly Upload $upload,
19     ){
20         parent::__construct($id, $title, $icon, $language, $twigEnvironment, $getTranslation);
21     }
22
23     public static function create(/* analogo al costruttore */): self
24     {
25         return new self(/* ... */);
26     }
27
28     protected function execute(RowId $rowId, Table $table): OperationResponse
29     {

```

```

30     $clientId = RfidReaderId::fromString($rowId->id);
31     $rfidReader = $this->rfidReaderRepository->getById($clientId);
32
33     if ($rfidReader->isMqtt()) {
34         try {
35             $thingPrincipals = $this->awsIotClient->listThingPrincipals($clientId);
36
37             if (count($thingPrincipals) === 0) {
38                 throw new Exception('No principals attached to the thing');
39             }
40
41             foreach ($thingPrincipals as $certificateArn) {
42                 $this->awsIotClient->detachPrincipalPolicy($certificateArn, AwsIotPolicy::CLIENT);
43                 $this->awsIotClient->detachThingPrincipal($clientId, $certificateArn);
44
45                 $certificateId = $this->getIdFromArn($certificateArn);
46                 $this->awsIotClient->deleteCertificate($certificateId);
47             }
48
49             $certificateName = $rfidReader->getCertificateName();
50
51             if ($certificateName !== null) {
52                 $this->userFilesFilesystem->delete(
53                     $this->upload->getDefaultFilePath(
54                         $this->licenseAndUser->actingUnderLicense(),
55                         FileUploadContext::AWS_IOT_CERTIFICATE,
56                         $certificateName,
57                     ),
58                 );
59             }
60
61             $this->awsIotClient->deleteThing($clientId);
62         } catch (Throwable $e) {
63             $this->globalMessage->insert(
64                 ($this->getTranslation)('rfid_failed_reader_delete', $this->language),
65                 GlobalMessageType::ERROR,
66             );
67             $this->logger->error(
68                 sprintf(
69                     'Error deleting reader %s: %s',
70                     $clientId->id->toString(),
71                     $e->getMessage(),
72                 ),
73             );
74
75             return new NotExecuted(null);
76         }
77     }

```

```

78
79     $this->commandBus->execute(
80         new RemoveReaderCommand(
81             $clientId,
82             $this->licenseAndUser->actingAsUser(),
83             $this->licenseAndUser->actingUnderLicense(),
84         ),
85     );
86
87     $this->globalMessage->insert(
88         ($this->getTranslation('rfid_reader_deleted', $this->language),
89         GlobalMessageType::OK,
90     );
91
92     return new Executed(
93         Action::DELETE_ROW,
94     );
95 }
96
97 protected function getAskConfirmationConfiguration(RowId $rowId): AskConfirmationConfiguration
98 {
99     return new AskConfirmationConfiguration(
100         ($this->getTranslation('rfid_delete_reader', $this->language, ['ReaderId' => $rowId->id]),
101     );
102 }
103
104 private function getIdFromArn(string $id): string
105 {
106     $arn = explode('/', $id);
107     $result = end($arn);
108     Assert::stringNotEmpty($result);
109
110     return $result;
111 }
112 }

```

6.3. Ricezione dei tag RFID letti

6.3.1. Worker

Come anticipato nella Sezione §5.1, i dati dei tag RFID letti dai reader vengono estratti in *pull* da AWS SQS tramite un **worker**. Una volta estratto un messaggio il *worker* lo distribuisce, in base al tipo di messaggio, ad un opportuno **handler**, che nel caso dei messaggi ricevuti dalla coda “rfid-reader-tag-events” è il RfidEventsMessageSerializer.

Di seguito viene mostrato come nella classe Container [10], ovvero il contenitore di dipendenze del backend di KanbanBOX che serve per gestire la *dependency injection*,

viene preparato tutto il necessario per costruire il *worker* dedicato alla lettura degli eventi RFID.

Vediamo che viene costruito un oggetto `ConsumeMessagesCommand` che è il comando, eseguito dal worker, che consente di estrarre i messaggi dalla coda SQS e di processarli; questo comando viene costruito passando un `RoutableMessageBus` che è un bus dei messaggi che consente di instradare i messaggi a degli handler specifici in base al loro tipo, e un *locator* (`RfidEventsReceiverLocator`) che rappresenta il ricevitore SQS associato alla coda da cui vogliamo estrarre i messaggi.

Abbiamo poi anche un `EventDispatcher` per gestire gli eventi generati durante l'esecuzione del comando, e un `Logger` per dare in *output* eventuali errori o messaggi di log.

Infine viene fornito anche il nome coda sotto forma di array, che in altri contesti potrebbe essere utile per gestire più code con lo stesso comando, ma nel nostro caso è un parametro superfluo dato che abbiamo un solo comando dedicato ad una sola coda, già associata al `RfidEventsReceiverLocator`.

Sarà il `ConsumeMessagesCommand` ad occuparsi della gestione dalla configurazione e del ciclo di vita del Worker [11], utilizzando i componenti forniti durante la configurazione del `ConsumeMessagesCommand` nel Container.

```
1  <?php
2  $rfidEventsQueueConfiguration = $getConfig->getRfidEventsQueueConfiguration();
3  $rfidEventsReceiverLocator = new EmptyContainer();
4  $rfidEventsReceiverLocator->set(
5      $rfidEventsQueueConfiguration->queueName,
6      $awsSqsFactory->buildReceiver(
7          $rfidEventsQueueConfiguration,
8          new RfidEventsMessageSerializer($clock, $logger),
9      ),
10 );
11
12 $rfidEventsCommandExecutor = new ConsumeMessagesCommand(
13     new RoutableMessageBus(
14         new EmptyContainer(),
15         $messageBus,
16     ),
17     $rfidEventsReceiverLocator,
18     $eventDispatcher,
19     $logger,
20     [$rfidEventsQueueConfiguration->queueName],
21 );
```

6.3.2. RfidEventsMessageSerializer

`RfidEventsMessageSerializer` è il componente che si occupa di **manipolare** i messaggi provenienti dalla coda SQS e di trasformarli in **DTO** che possono essere gestiti dal sistema di messaggistica interno. Nel flusso del *worker* mostrato nella sezione precedente, il serializer viene passato alla factory che costruisce il receiver SQS (`buildReceiver(...)`): in questo modo, ogni payload JSON letto dalla coda viene

convertito, tramite il serializer, in un `Envelope` contenente un oggetto di dominio, che verrà poi instradato dal `RoutableMessageBus` verso l'handler corretto.

I **parametri** passati al costruttore del serializer sono un `Clock` e un `LoggerInterface`: il primo viene usato per assegnare il timestamp agli eventi generati, mentre il secondo è utile per *loggere* eventuali errori o messaggi di diagnostica durante la decodifica dei messaggi.

Il metodo **`decode(...)`** è il componente principale del serializer, invocato per lo più dall'oggetto `AmazonSqsReceiver` [12] del framework `Symfony` quando viene letto un messaggio dalla coda `SQS`; il suo funzionamento è il seguente:

- se il `type` del messaggio è `heartbeat` e il `clientId` è presente, viene creato un `ReportEventsMessage` contenente un report di tipo `ReaderReportContainsHeartbeat`; il `Clock` viene usato per assegnare l'istante di ricezione e viene generato un identificativo univoco del report (`Uuid::uuid4()`).
- se il payload contiene l'id del tag letto (`idHex`) e il `clientId`, viene costruito un `ReadTagEventsMessage` con i dati della lettura (timestamp, RSSI, antenna, numero di letture, ecc.) e l'identificativo del reader che ha generato il messaggio, ricavato dal `clientId`;
- se la struttura non è valida o mancano campi obbligatori, viene emesso un warning e il serializer restituisce un `MessageToBeDiscarded`, così da evitare che il worker propaghi messaggi malformati nel resto del sistema.

L'oggetto **`Envelope` restituito** non è altro che un wrapper utile per eseguire la distribuzione del messaggio nel sistema di messaggistica interno (`MessageBus`).

È importante notare come venga definito uno **`psalm-type`** (`RfidMessageBody`) per il corpo del messaggio, di cui non è stata riportata la definizione completa per questioni di spazio e chiarezza; questa annotazione è utile per indicare a Psalm di imporre la tipizzazione statica specificata, sulle variabili a cui è assegnata, in questo caso `$decodedEnvelope`, evitando di dover gestire tipi di dati `mixed` o non conformi a quelli attesi.

```
1  <?php PHP
2  /** @psalm-type RfidMessageBody = array{...} */
3  final readonly class RfidEventsMessageSerializer implements SerializerInterface
4  {
5      public function __construct(
6          private Clock $clock,
7          private LoggerInterface $logger,
8      ){
9      }
10
11     /** @inheritdoc */
12     public function decode(array $encodedEnvelope): Envelope
13     {
14         /** @var RfidMessageBody $decodedEnvelope */
15         $decodedEnvelope = json_decode($encodedEnvelope['body'], true, 512, JSON_THROW_ON_ERROR);
16         if (($decodedEnvelope['type'] ?? null) === 'heartbeat' && isset($decodedEnvelope['clientId'])) {
17             return Envelope::wrap(
```

```

18         new ReportEventsMessage(
19             ReaderReportContainsHeartbeat::from(
20                 $this->clock->now(),
21                 [
22                     'rfidReport' => Uuid::uuid4()->toString(),
23                     'reader' => $decodedEnvelope['clientId'],
24                 ],
25             ),
26         ),
27     );
28 }
29
30 if (isset($decodedEnvelope['data']['idHex'], $decodedEnvelope['clientId'])) {
31     if($type->matches($decodedEnvelope)) {
32         return Envelope::wrap(new ReadTagEventsMessage(
33             MessageId::generate(),
34             $decodedEnvelope['data']['idHex'],
35             $decodedEnvelope['type'],
36             new DateTimeImmutable($decodedEnvelope['timestamp']),
37             $decodedEnvelope['data']['reads'],
38             $decodedEnvelope['data']['phase'],
39             $decodedEnvelope['data']['peakRssi'],
40             $decodedEnvelope['data']['format'],
41             $decodedEnvelope['data']['eventNum'],
42             $decodedEnvelope['data']['antenna'],
43             RfidReaderId::fromString($decodedEnvelope['clientId']),
44         ));
45     }
46
47     $this->logger->warning('Received a message with an unexpected format or missing required data', [
48         'category' => 'rfid',
49         'message' => $decodedEnvelope,
50     ]);
51 }
52
53 return Envelope::wrap(
54     new MessageToBeDiscarded(),
55 );
56 }
57 }

```

6.3.3. ReadTagEventsMessage

ReadTagEventsMessage è il DTO che rappresenta un singolo **tag RFID** letto, che è stato recuperato dalla coda SQS.

Come anticipato anche in §6.3.2., questo oggetto viene creato dal RfidEventsMessageSerializer quando il payload contiene i campi minimi necessari, ovvero l'identificativo del tag `idHex` e l'identificativo del reader `clientId`.

Di seguito vengono descritti i **campi** dell'oggetto:

- **id** è l'identificativo univoco del messaggio nel sistema di messaggistica interno (utile per tracciamento e diagnostica);
- **idHex** è l'identificativo del tag letto (tipicamente l'EPC in formato esadecimale) ed è l'informazione che permette di associare la lettura a un **kanban**;
- **now** rappresenta il timestamp della lettura (o, più in generale, l'istante associato all'evento di lettura che viene propagato nel dominio);
- **reads**, **phase**, **peakRssi**, **antenna**, **eventNum**, **format** rappresentano le statistiche relative alla lettura;
- **clientId** è l'identificativo del reader che ha prodotto l'evento.

La classe implementa i **metodi** dell' **interfaccia Message**:

- **getAttributes()** ritorna null perché la classe non aggiunge ulteriori metadati;
- **getBody()** serializza tutti i campi (per questioni di spazio e chiarezza non sono stati elencati tutti) in JSON per consentire log/diagnostica o trasporto interno coerente con gli altri Message.

```
1  <?php PHP
2  final readonly class ReadTagEventsMessage implements Message
3  {
4      public function __construct(
5          public MessageId $id,
6          public string $idHex,
7          public string $type,
8          public DateTimeImmutable $now,
9          public int $reads,
10         public float $phase,
11         public int $peakRssi,
12         public string $format,
13         public int $eventNum,
14         public int $antenna,
15         public RfidReaderId $clientId,
16     ){
17     }
18
19     public function getAttributes(): MessageAttributes|null
20     {
21         return null;
22     }
23
24     public function getBody(): string
25     {
26         return json_encode([
27             'id' => $this->id->__toString(),
28             'idHex' => $this->idHex,
29             // ... associazione chiave-valore di tutti i campi ...
30         ], JSON_THROW_ON_ERROR);
31     }
```


6.3.4. ReportEventsMessage e ReaderReportContainsHeartbeat

Nel caso dei messaggi di tipo **heartbeat**, l'obiettivo non è associare un tag a un kanban, ma aggiornare lo **stato di connessione** del reader nel backend. Per questo motivo il serializer, quando riceve un payload con `type = heartbeat` e con `clientId` presente, costruisce un messaggio di tipo `ReportEventsMessage`.

`ReportEventsMessage` è un DTO molto semplice pensato per essere compatibile con il **MessageBus** (per questo implementa `Message`) e *wrappare* l'evento `ReaderReportContainsHeartbeat`.

In questo modo è possibile trasmettere l'evento `ReaderReportContainsHeartbeat` (già esistente e adatto a questo caso d'uso) attraverso il sistema di messaggistica interno.

```

1  <?php
2  final readonly class ReportEventsMessage implements Message
3  {
4      /**
5       * DTO representing a heartbeat
6       */
7      public function __construct(
8          public ReaderReportContainsHeartbeat $readerReportContainsHeartbeat,
9      ){
10     }
11
12     public function getAttributes(): MessageAttributes|null
13     {
14         return null;
15     }
16
17     public function getBody(): string
18     {
19         return "";
20     }
21 }
```

`ReaderReportContainsHeartbeat` è invece un **domain event** (`AggregateDomainEvent`) che rappresenta un evento relativo ad uno specifico DTO, in questo caso un `RfidReport`, ovvero un messaggio diagnostico ricevuto da un reader RFID.

I parametri passati al **costruttore** sono:

- **report**: l'identificativo dell'`RfidReport` associato a questo evento;
- **reader**: l'identificativo del reader che ha generato l'heartbeat;
- **raisedAt**: l'istante in cui l'evento è stato generato.

Il metodo **raise(...)** è un *factory method* che consente di creare un'istanza di `ReaderReportContainsHeartbeat` a partire dai dati ricevuti dal serializer, e assegnando l'istante di generazione tramite il `Clock`.

I metodi **toArray()** e **from(...)** sono invece utili per serializzare e deserializzare l'evento, quindi per adattarlo ai formati necessari nei contesti in cui può essere utilizzato.

```
1  <?php
2  final class ReaderReportContainsHeartbeat implements AggregateDomainEvent
3  {
4      private function __construct(
5          private RfidReportId $report,
6          public RfidReaderId $reader,
7          private DateTimeImmutable $raisedAt,
8      ){
9      }
10
11     public static function raise(
12         RfidReportId $report,
13         RfidReaderId $reader,
14         Clock $clock,
15     ): self
16     {
17         return new self(
18             $report,
19             $reader,
20             $clock->now(),
21         );
22     }
23
24     /* ... getters ... */
25
26     /** @return array<string, string> */
27     public function toArray(): array
28     {
29         return [
30             'rfidReport' => $this->aggregate()->toString(),
31             'reader' => $this->reader->id->toString(),
32         ];
33     }
34
35     public static function from(
36         DateTimeImmutable $raisedAt,
37         array $data,
38     ): self
39     {
40         return new self(
41             RfidReportId::fromString($data['rfidReport']),
42             RfidReaderId::fromString($data['reader']),
43             $raisedAt,
44         );
```

PHP

```

45 }
46 }

```

6.3.5. RfidEventsMessageHandler, UpdateLastHeartbeatOfTheReaderCommand e RfidScan

RfidEventsMessageHandler è l'*handler* che riceve i messaggi incapsulati dal RfidEventsMessageSerializer e li usa per **riflettere i cambiamenti** indicati dai messaggi stessi nel dominio, ad esempio aggiornando lo stato di un reader o registrando una lettura di un tag RFID.

Nel flusso del *worker* descritto in precedenza, dopo la decodifica e il wrapping in un Envelope, il RoutableMessageBus instrada i messaggi, di tipo esplicitamente ammesso (nel Container le classi di messaggi vengono associate agli *handler* corrispondenti), verso questo handler.

Il **costruttore** riceve una serie di **dipendenze** che permettono di applicare la logica di business e la persistenza dei dati:

- **RfidScanRepository**: oggetto per gestire la persistenza dei dati a DB, associato specificatamente alla tabella `rfid_scan` che in questo caso ci servirà per gli eventi di lettura dei tag RFID;
- **IgnoreScanDueToDebouncing**: oggetto che implementa il **debouncing** per evitare di registrare letture duplicate troppo ravvicinate;
- **RfidAreaByReaderIdAndAntennaName**: oggetto che quando invocato permette di ottenere dal DB l'area associata ad una specifica combinazione di reader e antenna, così da poter associare la lettura del tag a un'area specifica;
- **StringIdFromEpc**: oggetti per la conversione dell'EPC (identificativo del tag fisico) da esadecimale a stringa usata per identificare i cartellini internamente a KanbanBOX;
- **FindCardFromStringId**: oggetto che permette di cercare un cartellino a DB usando il suo identificativo;
- **Clock**: oggetto per la gestione di timestamp o data e ora, utile per assegnare il timestamp alle letture o agli eventi generati;
- **CommandBus**: oggetto che permette di eseguire i comandi costruiti nel metodo `handle(...)`.

Il metodo **handle(...)** gestisce due macro-casi, coerenti con i due DTO introdotti nelle sezioni precedenti:

- **heartbeat** (ReportEventsMessage): evento diagnostico che segnala che un reader è connesso e attivo;
- **tag scan** (ad es. ReadTagEventsMessage): evento di lettura di un tag RFID.

```

1  <?php
2  final readonly class RfidEventsMessageHandler implements MessageHandler
3  {
4      public function __construct(
5          private RfidScanRepository $rfidScanRepository,
6          private IgnoreScanDueToDebouncing $ignoreScanDueToDebouncing,
7          private RfidAreaByReaderIdAndAntennaName $rfidAreaIdByReaderIdAndAntennaName,
8          private StringIdFromEpc $stringIdFromEpc,

```

PHP

```

9     private FindCardFromStringId $findCardFromStringId,
10    private Clock $clock,
11    private CommandBus $commandBus,
12   ){
13   }
14
15   public function handle(Message $message): void
16   {
17       if ($message instanceof ReportEventsMessage) {
18           $this->commandBus->execute(
19               UpdateLastHeartbeatOfTheReaderCommand
20               ::fromReaderReportContainsHeartbeat(
21                   $message->readerReportContainsHeartbeat,
22                   ),
23           );
24
25       return;
26   }
27
28   $event = RfidScan::validateScan(
29       $message->clientId,
30       $message->idHex,
31       $message->peakRssi,
32       (string) $message->antenna,
33       $message->now,
34       $this->ignoreScanDueToDebouncing,
35       $this->rfidAreaIdByReaderIdAndAntennaName,
36       $this->stringIdFromEpc,
37       $this->findCardFromStringId,
38       $this->clock,
39   );
40   $this->rfidScanRepository->store($event);
41   }
42   }

```

Nel caso dell'**heartbeat**, l'handler traduce l'evento in un comando **UpdateLastHeartbeatOfTheReaderCommand** usando i dati contenuti nel `ReaderReportContainsHeartbeat` *wrappato* nel messaggio. Il `CommandBus` eseguirà il comando, aggiornando, con la data e ora dell'heartbeat appena ricevuto, la colonna `last_heartbeat` nella tabella `rfid_reader`.

```

1  <?php
2  final class UpdateLastHeartbeatOfTheReaderCommand implements RfidScanCommand
3  {
4      private function __construct(public RfidReaderId $reader, public DateTimeImmutable $raisedAt)
5      {
6      }
7

```

PHP

```

8     public static function fromReaderReportContainsHeartbeat(ReaderReportContainsHeartbeat $event): self
9     {
10         return new self($event->reader, $event->raisedAt());
11     }
12
13     public function credentials(): System
14     {
15         return new System();
16     }
17 }

```

Invece, per la **lettura dei tag**, l'handler delega la validazione e la costruzione dell'evento a **RfidScan::validateScan(...)**, passando:

- i dati contenuti nel tag (clientId, idHex, RSSI, antenna, timestamp);
- le dipendenze necessarie a validare correttamente la lettura del tag.

Il risultato della validazione è un evento di lettura di un tag (RfidScan) di cui si garantisce la correttezza dei dati contenuti e a cui è stata associata l'area in cui è stato letto.

Come spiegato alla fine di §5.1.1., associando i tag letti all'area corretta possiamo poi identificare il cambio stato previsto dall'area e **aggiornare** di conseguenza il **cartellino kanban** con identificativo corrispondente a quello del tag RFID letto.

```

1  <?php
2  final class RfidScan implements AggregateRoot
3  {
4      private RfidScanId $id;
5
6      /** @var list<AggregateDomainEvent<self>> */
7      public array $newEvents = [];
8
9      /** @var positive-int|0 */
10     private int $version = 0;
11
12     private function __construct(RfidScanId $id)
13     {
14         $this->id = $id;
15     }
16
17     public static function validateScan(
18         RfidReaderId $readerId,
19         string $epc,
20         int $peakRssi,
21         string $antennaName,
22         DateTimeImmutable $scanTime,
23         IgnoreScanDueToDebouncing $ignoreScanDueToDebouncing,
24         RfidAreaByReaderIdAndAntennaName $areaByReaderIdAndAntennaName,
25         StringIdFromEpc $stringIdFromEpc,

```

```

26     FindCardFromStringId $findCardFromStringId,
27     Clock $clock,
28 ): self
29 {
30     $instance = new self(RfidScanId::generate());
31
32     // Validate the RFID EPC
33     try {
34         $cardStringId = $stringIdFromEpc($epc);
35     } catch (InvalidRfidEpc) {
36         $instance->newEvents[] = ScanDiscardedForInvalidRfidEpc::raise($instance->id, $readerId, $epc,
37             $clock);
38     }
39     return $instance;
40
41     // Identify the area
42     $area = $areaByReaderIdAndAntennaName($readerId, $antennaName);
43
44     // Validate area existence
45     if ($area === null) {
46         $instance->newEvents[] = ScanDiscardedForNonExistingAntennaByName::raise($instance->id,
47             $readerId, $antennaName, $clock);
48     }
49     return $instance;
50
51     // Validate the area is active
52     if (!$area->isActive()) {
53         $instance->newEvents[] = ScanDiscardedForAntennasAreaNotActive::raise($instance->id, $area-
54             >getId(), $epc, $antennaName, $cardStringId, $clock);
55     }
56     return $instance;
57
58     // Ignore the scan if it's due to debounce
59     if ($ignoreScanDueToDebouncing($area->getId(), $epc, $area->getDebouncingSeconds())) {
60         $instance->newEvents[] = ScanIgnoredDueToDebouncing::raise($instance->id, $area->getId(), $epc,
61             $peakRssi, $scanTime, $clock);
62     }
63     return $instance;
64
65     // Associate the scan to the area
66     $instance->newEvents[] = ScanAssociatedToArea::raise($instance->id, $area->getId(), $epc, $peakRssi,
67         $scanTime, $cardStringId, $clock);
68
69     // Associate the scan to the card
70     try {

```

```

70     $cardData = $findCardFromStringId($area->getLicense()->getId(), $cardStringId);
71
72     $instance->newEvents[] = ScanAssociatedToCard::raise(
73         $instance->id,
74         $area->getId(),
75         $sepc,
76         $cardStringId,
77         $cardData['cardId'],
78         $clock,
79     );
80     } catch (CardNotFound) {
81     }
82
83     return $instance;
84 }
85
86 /* ... metodi ausiliari ... */
87 }

```

Capitolo 7.

Verifica e validazione

In questo capitolo si parlerà anche dell'implementazione dei test di unità e integrazione per le funzionalità implementate, ma è importante premettere che, in accordo con il tutor interno, si è deciso di **dare priorità dell'implementazione delle funzionalità** in modo da poterle completare nei tempi previsti; per questo motivo i test implementati coprono solamente le classi strettamente legate al dominio di KanbanBOX, come ad esempio `RfidEventsMessageSerializer` o `RfidEventsMessageHandler`, mentre non sono stati implementati i test di classi strettamente legate a servizi esterni su cui si ha meno controllo, come `AwsIotClientImplementation` che, interagendo con le API di AWS, risulta molto più oneroso da testare.

Nonostante questo, si può affermare che le classi nel dominio di KanbanBOX sono state esaustivamente testate, dato che la CI [§7.4.] verifica che le classi in questione siano state correttamente testate.

7.1. Analisi statica del codice

Per aumentare l'affidabilità del codice e ridurre i difetti intercettabili prima dell'esecuzione, nel progetto sono stati introdotti strumenti di **analisi statica** che possono essere eseguiti arbitrariamente, in locale, tramite target dedicati del Makefile. Questi controlli sono integrati anche nella pipeline di CI [§7.4.] e vengono eseguiti automaticamente.

Gli strumenti principali utilizzati sono:

- **PHPCS** (PHP Coding Standards): effettua il controllo di **conformità allo standard di codifica** e individua violazioni di stile basandosi su regole di qualità configurate nel progetto, come naming, spaziature, indentazione, ecc. .
Nel progetto `phpcs` **non modifica** i file ma si limita a produrre un report e a far fallire l'esecuzione (e la CI) se il numero o la severità delle violazioni supera quanto consentito dalla configurazione.
- **PHPCBF** (PHP Code Beautifier and Fixer): è il «**complemento correttivo**» di PHPCS e applica le correzioni che possono essere automatizzate. In pratica, quando eseguito tramite `make phpcbf`, può **modificare direttamente i file sorgenti** normalizzando aspetti come indentazione, rimozione di trailing whitespace, aggiunta del newline finale e altre correzioni a non distruttive.
- **Psalm**: esegue analisi statica sul codice PHP con l'obiettivo di individuare **potenziali bug e incoerenze di tipizzazione**. Basandosi su tipi nativi e annotazioni PHPDoc (ad es. `@psalm-type` mostrato in §6.3.2.), Psalm segnala problemi come: tipi incompatibili tra argomenti e parametri, valori `null` non gestiti, return type errati o mancanti e altri problemi che in PHP emergerebbero solo a esecuzione.

Psalm **non applica modifiche** al codice, ma fallisce la validazione se trova errori oltre le soglie configurate.

7.2. Implementazione dei test di unità

I test di unità del progetto sono stati implementati principalmente tramite **PHPUnit**, con classi di test che estendono **TestCase**. Lo scopo del test di unità è verificare il comportamento di una singola unità (classe o metodo) **in isolamento** dal resto del sistema, rendendo i test veloci, deterministici e semplici da eseguire in locale e in CI.

Il blocco di codice seguente mostra un esempio reale di test di unità per **DownloadReaderCertificateRow**, ovvero una **row operation** della tabella di gestione dei reader RFID (descritta nella sezione §6.2.7.) e verrà sfruttato per descrivere, in modo generico, il processo di implementazione dei test di unità adottato in KanbanBOX.

Nel dettaglio, questi sono i **passaggi** seguiti per implementare i test di unità:

- **inizializzazione comune tramite setUp()**: i test creano le dipendenze condivise (mock e helper) una sola volta; nell'esempio vengono creati i mock di **TwigEnvironment**, **InternalUrlBuilder** e **GetTranslationMock**, poi viene inizializzato **TestingHelper**, una classe di supporto per semplificare la costruzione di oggetti usati frequentemente;
- **preparazione**: si costruisce l'oggetto da testare e si prepara l'input del caso d'uso (ad es. **RowId** e **Table**), configurando i mock in base allo scenario;
- **esecuzione**: si invoca il metodo sotto test; nel caso di una **row operation** questo avviene tramite **handle(...)**, che incapsula la logica di conferma e delega al metodo **execute(...)** solo quando i parametri indicano che l'utente ha confermato;
- **assert**: si verifica l'output tramite asserzioni, cioè metodi che controllano che il risultato dell'esecuzione sia conforme a quanto atteso.

Un aspetto centrale nell'approccio ai test unitari è l'uso dei **mock** per sostituire dipendenze esterne o non rilevanti per quel test. Nel frammento in esempio, **TwigEnvironment** e **InternalUrlBuilder** vengono sostituiti tramite **\$this->createMock(...)**: questo consente di iniettare dipendenze valide senza dover predisporre un sistema di template reale o una configurazione completa di routing.

PHPUnit espone due modalità principali quando si lavora con un mock:

- **stub**: si imposta un valore di ritorno per simulare un comportamento (es. **method(...)->willReturn(...)**);
- **expectation**: si verifica che un metodo venga chiamato un certo numero di volte e/o con certi argomenti (es. **expects(\$this->once())->method(...)->with(...)**).

Nell'esempio la combinazione tra **expectation** e **stub** viene usata per controllare l'interazione con **InternalUrlBuilder**: nel test **testExecute()** ci si aspetta che **build(...)** venga chiamato **una sola volta** con un path costruito a partire dal **RowId**, e si imposta il valore di ritorno per rendere l'esito deterministico:

```
1 $this->urlBuilder
2   ->expects(self::once())
3   ->method('build')
4   ->with('rfid/download_certificate/' . $rowId->id)
5   ->willReturn($expectedUrl);
```

PHP

Nel test `testExecuteWithoutConfirmation()` invece si imposta `expects(self::never())` per verificare che, in assenza di conferma, l'URL non venga nemmeno costruito (quindi non vengano eseguite operazioni non necessarie).

Per verificare l'esito, vengono usate **asserzioni** fornite da `TestCase`. Nel codice si vede l'uso di:

- `self::assertEquals(...)`, usato per confrontare oggetti risposta complessi (ad es. `RedirectToLink`) con un valore atteso costruito dal test;
- `self::assertInstanceOf(...)`, utile quando è sufficiente verificare **il tipo** del risultato (ad es. `AskConfirmation` quando manca la conferma) senza legarsi ai dettagli interni dell'oggetto.

In altri casi (non mostrati) è comune usare anche `assertSame` (identità), `assertTrue/assertFalse` (condizioni) o `expectException` (verifica di eccezioni).

Attenzione, nel frammento compare anche l'uso di `assert(...)` del linguaggio (non di PHPUnit): in questo contesto viene usato soprattutto come supporto alla **tipizzazione** e all'analisi statica, e non come meccanismo di verifica del test.

```
1  <?php PHP
2  #[CoversClass(DownloadReaderCertificateRow::class)]
3  class DownloadReaderCertificateRowTest extends TestCase
4  {
5      /** @var TwigEnvironment&MockObject */
6      private TwigEnvironment $twigEnvironment;
7
8      /** @var InternalUrlBuilder&MockObject */
9      private InternalUrlBuilder $urlBuilder;
10
11     private GetTranslationMock $getTranslation;
12     private TestingHelper $testingHelper;
13
14     protected function setUp(): void
15     {
16         $this->twigEnvironment = $this->createMock(TwigEnvironment::class);
17         $this->urlBuilder      = $this->createMock(InternalUrlBuilder::class);
18
19         $this->getTranslation = new GetTranslationMock();
20         $this->testingHelper  = new TestingHelper();
21     }
22
23     public function testExecute(): void
24     {
25         $downloadReaderCertificateRow = DownloadReaderCertificateRow::create(
26             'download_certificate',
27             'Download Certificate',
28             'download',
29             Language::EN,
30             $this->getTranslation,
```

```

31     $this->twigEnvironment,
32     $this->urlBuilder,
33 );
34
35 $rowId = new RowId('123');
36 $table = $this->testingHelper->buildTable('readerId');
37
38 $expectedUrl = new Uri('http://example.com/rfid/download_certificate/123');
39 $this->urlBuilder
40     ->expects(self::once())
41     ->method('build')
42     ->with('rfid/download_certificate/' . $rowId->id)
43     ->willReturn($expectedUrl);
44
45 $response = $downloadReaderCertificateRow->handle($rowId, ['confirm' => 'yes'], $table);
46
47 $expectedResponse = new RedirectToLink($expectedUrl, Target::NewTab, reloadRow: true);
48 self::assertEquals($expectedResponse, $response);
49 }
50
51 public function testExecuteWithoutConfirmation(): void
52 {
53     $downloadReaderCertificateRow = DownloadReaderCertificateRow::create(
54         'download_certificate',
55         'Download Certificate',
56         'download',
57         Language::EN,
58         $this->getTranslation,
59         TwigEnvironmentMock::createFakeEnvironment([
60             'CustomTable/Body/Row/Operation/askConfirmation.twig'],
61         [
62             [
63                 'messages' => ['rfid_download_certificate_confirmation_[]_en_'],
64                 'buttons' => [
65                     ['name' => 'general_confirm_[]_en_', 'icon' => 'tick', 'value' => 'yes'],
66                     ['name' => 'gen_cancel_[]_en_', 'icon' => 'cross', 'value' => 'cancel'],
67                 ],
68             ],
69         ],
70     ),
71     $this->urlBuilder,
72 );
73
74 $rowId = new RowId('123');
75
76 $this->urlBuilder->expects(self::never())->method('build');
77

```

```

78     $response = $downloadReaderCertificateRow->handle($rowId, [], $this->testingHelper-
    >buildTable('readerId'));
79
80     self::assertInstanceOf(AskConfirmation::class, $response);
81 }
82 }

```

7.3. Implementazione dei test di integrazione

I test di integrazione hanno l'obiettivo di verificare che **più componenti collaborino correttamente** tra loro, includendo tipicamente la persistenza su database o l'uso di servizi esterni reali.

Il frammento di codice seguente mostra un esempio di test di integrazione per `RfidEventsMessageHandler`, che gestisce i messaggi RFID decodificati dal worker. Il test estende `IsolatedDatabaseTransactionTestCase`: questa classe base prepara un ambiente di test con un **container** disponibile (`$this->container`) e un database eseguito in maniera **isolata**, tipicamente tramite una transazione che viene ripristinata a fine test. In questo modo ogni test può leggere/scrivere su DB senza «sporcare» lo stato per i test successivi.

Nel metodo `setUp()` si nota un pattern ricorrente:

- si invocano le operazioni di setup della classe base (`parent::setUp()`);
- si recuperano dal container implementazioni reali (in questo caso `RfidScanRepository`, `RfidAreaByReaderIdAndAntennaName`, `FindCardFromStringId`, `Clock`);
- si decide in modo esplicito cosa inizializzare tramite **mock**; in questo caso `CommandBus` viene sostituito con un mock perché l'interesse del test è verificare che l'handler **invochi** il comando corretto.

Anche nei test di integrazione rimangono validi i passaggi mostrati per i test di unità:

- **preparazione**: si costruisce l'input del caso d'uso, che in questo caso è un messaggio di tipo `ReadTagEventsMessage` o `ReportEventsMessage` (a seconda del caso), e si configurano i mock se necessario;
- **esecuzione**: si invoca il metodo `handle(...)` dell'handler, che rappresenta il punto di ingresso per i messaggi decodificati dal worker;
- **assert**: si verifica che il risultato dell'esecuzione sia quello atteso.

Anche in questa tipologia di test possono essere sfruttate le modalità **stub** ed **expectation** dei mock, spiegati nel capitolo §7.2., ad esempio per verificare che il `CommandBus` venga invocato con il comando corretto quando si gestisce un messaggio di tipo **heartbeat**.

```

1  <?php
2  #[CoversClass(ReadTagEventsMessage::class)]
3  #[CoversClass(RfidEventsMessageHandler::class)]
4  class RfidEventsMessageHandlerTest extends IsolatedDatabaseTransactionTestCase
5  {

```

PHP

```

6     private RfidScanRepository $rfidScanRepository;
7     private RfidAreaByReaderIdAndAntennaName $rfidAreaIdByReaderIdAndAntennaName;
8     private IgnoreScanDueToDebouncing $ignoreScanDueToDebouncing;
9     private FindCardFromStringId $findCardFromStringId;
10    private StringIdFromEpc $stringIdFromEpc;
11    private Clock $clock;
12    private RfidEventsMessageHandler $rfidEventsMessageHandler;
13
14    /** @var CommandBus&MockObject */
15    private CommandBus $commandBus;
16
17    public function setUp(): void
18    {
19        parent::setUp();
20
21        $this->rfidScanRepository = $this->container->get(RfidScanRepository::class);
22        $this->rfidAreaIdByReaderIdAndAntennaName = $this->container->get(RfidAreaByReaderIdAndAntennaName::class);
23        $this->ignoreScanDueToDebouncing = $this->container->get(IgnoreScanDueToDebouncing::class);
24        $this->stringIdFromEpc = new DefaultStringIdFromEpc();
25        $this->findCardFromStringId = $this->container->get(FindCardFromStringId::class);
26        $this->clock = $this->container->get(Clock::class);
27        $this->commandBus = $this->createMock(CommandBus::class);
28
29        $this->rfidEventsMessageHandler = new RfidEventsMessageHandler(
30            $this->rfidScanRepository,
31            $this->ignoreScanDueToDebouncing,
32            $this->rfidAreaIdByReaderIdAndAntennaName,
33            $this->stringIdFromEpc,
34            $this->findCardFromStringId,
35            $this->clock,
36            $this->commandBus,
37        );
38    }
39
40    public function testHandlerWithRfidScan(): void
41    {
42        $readTagEventsMessage = new ReadTagEventsMessage(
43            MessageId::fromString('1f11d640-b2c8-6772-b39d-4202f28cd586'),
44            '3034257BF461AABDD0000001',
45            'type',
46            new DateTimeImmutable('2026-02-25T12:00:00Z'),
47            99,
48            1.0,
49            27,
50            'format',
51            1,

```

```

52     1,
53     RfidReaderId::fromString('1f11d640-b2c8-6772-b39d-4202f28cd586'),
54 );
55
56 $this->rfdEventsMessageHandler->handle($readTagEventsMessage);
57
58 $storedScan = $this->rfdScanRepository->get(RfidScanId::fromString('1f11d640-b2c8-6772-
59 b39d-4202f28cd586'));
60 self::assertEquals($readTagEventsMessage->clientId->id, $storedScan->aggregateRootId()->toString());
61 }
62
63 public function testHandlerWithReportEventsMessage(): void
64 {
65     $reportEventsMessage = new ReportEventsMessage(
66         ReaderReportContainsHeartbeat::from(
67             new DateTimeImmutable('2026-02-25T12:00:00Z'),
68             [
69                 'rfidReport' => '1f11d640-b2c8-6772-b39d-4202f28cd586',
70                 'reader' => '123a4567-b2c8-6772-b39d-4202f28cd586',
71             ],
72         ),
73         $updateLastHeartbeatOfTheReaderCommand =
74             UpdateLastHeartbeatOfTheReaderCommand::fromReaderReportContainsHeartbeat(
75                 $reportEventsMessage->readerReportContainsHeartbeat,
76             );
77     $this->commandBus->expects(self::once())
78         ->method('execute')
79         ->with($updateLastHeartbeatOfTheReaderCommand);
80
81     $this->rfdEventsMessageHandler->handle($reportEventsMessage);
82 }
83 }

```

7.4. Continuous Integration

Viene citata la **Continuous Integration** (CI) in questo capitolo perché è proprio tramite le Github Actions [§2.2.1.] che vengono eseguite le operazioni di verifica e validazione prima che il codice venga inserito in produzione.

Nella CI implementata da KanbanBOX vengono eseguiti i workflow relativi a tutti gli strumenti di analisi statica mostrati in §7.1., e successivamente anche i test di unità e integrazione descritti nei capitoli precedenti.

Oltre a questi controlli, la CI esegue anche altri workflow relativi a strumenti che non sono stati affrontati in modo rilevanti durante questo progetto; tra questi risulta interessante menzionare:

- **infection**, strumento per l'analisi della **mutazione** del codice, che verifica l'efficacia dei test esistenti introducendo modifiche graduali al codice e controllando se i test rispondono alle variazioni;
- **visual test** tramite **Playwright**, che consente di verificare il funzionamento dei componenti dell'interfaccia utente.

Capitolo 8.

Conclusioni

8.1. Consuntivo finale

Lo stage si è svolto in un periodo di circa 2 mesi, per un totale di ~316 ore di lavoro. Il progetto è stato portato a termine con successo; come mostrato in §3.4., tutti gli obiettivi prefissati sono stati raggiunti, ad eccezione dei requisiti:

- **R-01-Q-O**: test e validazione funzionale del driver sviluppato, raggiunto **parzialmente** data la mancanza di tempo per esplorare, partendo quasi da zero, lo stack necessario all'implementazione di test di integrazione più complessi, che avrebbero dovuto interagire con sistemi esterni su cui si ha un controllo ridotto come AWS IoT e i reader RFID.
- **R-02-Q-D**: implementazione di meccanismi atti a garantire la sicurezza della comunicazione, raggiunto **parzialmente** in quanto per questioni di tempo non è stato possibile implementare dei meccanismi di *logging* completamente esaustivi.

Il programma settimanale definito nel «Piano di Lavoro» è stato seguito in maniera abbastanza fedele, con alcune variazioni relative principalmente ai tempi necessari per lo svolgimento delle attività, che nel complesso si sono compensate, portando il numero di ore totali ad essere in linea con quanto previsto.

8.2. Conoscenze acquisite

Un punto forte di questo progetto di stage è stata l'**ampiezza** dei concetti approfonditi e delle tecnologie utilizzate, che mi ha permesso di sperimentare con nuovi strumenti o di consolidare conoscenze già acquisite precedentemente. Infatti ho avuto modo di operare nei seguenti ambiti:

- **infrastruttura cloud**, in particolare AWS, con servizi come AWS IoT Core, AWS SQS e AWS IAM;
- **dispositivi e ambienti IoT**, in particolare i reader RFID, con cui è spesso difficile avere un contatto diretto e pratico quando si lavora in ambiti accademici o personali;
- **protocolli di comunicazione**, in particolare MQTT, con cui non avevo avuto modo di lavorare prima;
- **strumenti di sviluppo**, come PHPStorm, Docker, Github e i vari framework di testing e supporto allo sviluppo, strumenti con cui avevo già avuto modo di lavorare, ma su cui ora ho maggiore padronanza;
- **sviluppo in PHP**, linguaggio che avevo già utilizzato in diverse occasioni ma che, assieme all'utilizzo di librerie e metodologie di sviluppo più avanzate, mi ha permesso di conoscere meglio le dinamiche di progetti a livello *enterprise*.

8.3. Sviluppi futuri

8.3.1. Rotazione certificati

Un aspetto importante da considerare per avere una piattaforma manutenibile e allo stesso tempo sicura è la **rotazione dei certificati**, ovvero la possibilità di sostituire

arbitrariamente i certificati utilizzati per l'autenticazione e la cifratura della comunicazione.

Attualmente, i certificati utilizzati dai reader RFID vengono generati e configurati automaticamente da KanbanBOX, richiedendo poi l'intervento manuale dell'utente per l'upload del certificato sul reader stesso tramite l'interfaccia web fornita dal produttore. Inoltre i certificati generati sono scaricabili solamente una volta tramite interfaccia di KanbanBOX, e non è possibile rigenerarli o recuperarli in caso di smarrimento.

L'implementazione ideale prevederebbe la possibilità di **rigenerare i certificati su AWS tramite KanbanBOX**, con la conseguente **gestione** dello storico dei certificati **eseguita** direttamente **dal sistema**, senza che l'utente debba preoccuparsi di gestirli manualmente, se non per caricarli sui reader RFID. In questo modo, in caso di smarrimento o compromissione dei certificati, sarebbe possibile rigenerarli in maniera semplice e veloce, senza dover ricreare i reader su KanbanBOX o dover operare manualmente nella console di AWS IoT.

8.3.2. Configurazione di un dominio custom per il broker MQTT

Attualmente il dominio dell'endpoint utilizzato per connettersi al broker MQTT di AWS IoT Core è quello di default fornito da AWS, che è un dominio generico che non riconduce in alcun modo a KanbanBOX.

Per questioni di **eleganza e di fiducia da parte dei clienti** sarebbe utile poter utilizzare un **sotto-dominio custom** del dominio di KanbanBOX già esistente, come ad esempio `iot.kanbanbox.com`, che riconduca direttamente al broker MQTT di AWS IoT Core.

8.3.3. Approfondimento della configurazione della coda SQS

Attualmente la coda SQS utilizzata per il trasferimento dei tag letti e degli heartbeat è configurata in modo abbastanza semplice e standard. Approfondire i concetti riguardanti i parametri di configurazione della coda potrebbe essere utile per **migliorare prestazioni e affidabilità** di tutto il sistema.

8.3.4. Test di integrazione

Come accennato in §3.4. e in §8.1., non è stato possibile implementare dei **test di integrazione** più complessi che avrebbero dovuto interagire con **sistemi esterni** come AWS IoT e i reader RFID.

In accordo con il tutor interno si è deciso di considerare comunque il progetto come completo considerando il livello di complessità che avrebbero aggiunto queste implementazioni. Inoltre ci si può comunque affidare a dei test di unità abbastanza esaustivi e al fatto che i servizi esterni utilizzati sono strumenti professionali testati e con un buon livello di affidabilità.

Nonostante sui servizi esterni si abbia anche un controllo ridotto, sarebbe comunque una buona pratica implementare dei test di integrazione che interagiscano con essi, in modo da poter **verificare** che la **gestione di errori o comportamenti anomali**, seppur rari, funzioni correttamente.

8.4. Considerazioni finali

Dal punto di vista personale credo che questo progetto mi abbia permesso di crescere molto come professionista del settore.

Ho avuto modo di affrontare concetti e sfide reali in un ambiente che mi ha fornito tutti gli strumenti e il supporto per operare al meglio, e mi ha permesso di avere un riscontro diretto sia vedendo il prodotto crescere anche grazie al mio contributo, sia interfacciandomi con i colleghi e con il tutor interno.

Penso che avere modo di lavorare su un progetto reale, con tecnologie che rappresentano uno standard del settore, compreso l'hardware a cui è difficile avere accesso se non in specifici contesti aziendali, sia un'opportunità preziosa e da non dare per scontata per chiunque voglia intraprendere una carriera in questo ambito.

Bibliografia

- [1] «AWS EC2 Pricing». [Online]. Disponibile su: <https://aws.amazon.com/ec2/pricing/on-demand/>
- [2] «AWS IoT Pricing». [Online]. Disponibile su: <https://aws.amazon.com/iot-core/pricing/>
- [3] «PHP MQTT Client Library». [Online]. Disponibile su: <https://github.com/php-mqtt/client>
- [4] «phpMQTT Client Library». [Online]. Disponibile su: <https://github.com/bluerhinos/phpMQTT>
- [5] «Simps MQTT Client Library». [Online]. Disponibile su: <https://github.com/simps/mqtt>
- [6] «AWS IoT SDK for PHP». [Online]. Disponibile su: <https://docs.aws.amazon.com/aws-sdk-php/v3/api/class-Aws.Iot.IotClient.html>
- [7] «AWS SQS SDK for PHP». [Online]. Disponibile su: <https://docs.aws.amazon.com/aws-sdk-php/v3/api/class-Aws.Sqs.SqsClient.html>
- [8] «Stati Kanban». [Online]. Disponibile su: <https://help.kanbanbox.com/hc/it/articles/360006520694-Stati-del-Kanban>
- [9] «AWS IoT SQL Reference». [Online]. Disponibile su: <https://docs.aws.amazon.com/iot/latest/developerguide/iot-sql-reference.html>
- [10] «PSR Container». [Online]. Disponibile su: <https://www.php-fig.org/psr/psr-11/>
- [11] «Symfony Worker». [Online]. Disponibile su: <https://github.com/symfony/symfony/blob/8.1/src/Symfony/Component/Messenger/Worker.php>
- [12] «Symfony AmazonSqsReceiver». [Online]. Disponibile su: <https://github.com/symfony/amazon-sqs-messenger/blob/8.1/Transport/AmazonSqsReceiver.php>

Glossario

A

API

Application Programming Interface, è un insieme di regole e protocolli che consentono a diverse applicazioni software di comunicare tra loro.

Le API definiscono i metodi e i formati per trasmettere richieste e risposte tra componenti software o hardware, facilitando la comunicazione tra sistemi diversi.

B

Backend

Il backend è la parte di un sistema software che gestisce la logica di funzionamento del sistema, l'elaborazione dei dati e l'interazione con il database o altri servizi. Non include, invece, l'interfaccia con cui l'utente finale interagisce o la parte visibile del sistema, che è chiamata frontend.

Nel caso di KanbanBox il backend viene eseguito su una macchina server, ovvero un dispositivo diverso e fisicamente distante dalla macchina usata dall'utente finale per interagire con il sistema.

Backup

Il backup è una copia esatta di dati, in un formato qualsiasi (compatibile con il formato dei dati originari), che viene creata per proteggere le informazioni in caso di perdita, danneggiamento o malfunzionamento del sistema originale.

Broker

Un broker è un componente software che funge da intermediario nella comunicazione tra dispositivi in un'architettura publish/subscribe, come quella utilizzata dal protocollo MQTT.

C

CLI

Command Line Interface, è un'interfaccia utente che consente agli utenti di interagire con un sistema software o hardware tramite comandi testuali lanciati in una console o terminale.

In questo documento viene usata anche per riferirsi agli strumenti utilizzabili da riga di comando, come ad esempio la CLI di AWS.

D

DTO

Data Transfer Object, è un oggetto che permette di standardizzare l'accesso e il trasferimento, tra componenti software diversi, di dati che si vuole organizzare seguendo una struttura ben definita, che può essere rappresentata da una classe o da un'interfaccia. Spesso i DTO garantiscono anche di imporre vincoli di tipo sui dati (parametri di funzione, tipi di ritorno, ecc.), vantaggio ampiamente sfruttato in questo progetto.

E

F

G

H

HTTP

Hypertext Transport Protocol, è un protocollo di comunicazione basato su architettura client-server, utilizzato principalmente per la trasmissione di dati sul web.

È il protocollo che è stato utilizzato fin dall'inizio in KanbanBOX per la comunicazione tra i lettori RFID e la piattaforma stessa.

IaaS

Infrastructure as a Service, è un modello di servizio cloud che fornisce risorse di calcolo virtualizzate su richiesta, come server, storage e reti, consentendo agli utenti di gestire e controllare l'infrastruttura sottostante senza doversi preoccupare della gestione fisica dell'hardware.

I

IDE

Integrated Development Environment, è un software che fornisce strumenti e funzionalità per facilitare lo sviluppo di applicazioni software, come un editor di codice, un compilatore, un debugger e altre funzionalità utili per la scrittura, il test e il debug del codice.

J

JSON

JavaScript Object Notation, è un formato di file testuali utilizzato per l'archiviazione e lo scambio di dati, tramite sistemi informatici, che rimane comunque leggibile anche da esseri umani.

K

Kanban

Kanban è un sistema di gestione della produzione e del flusso di lavoro che utilizza schede visive (chiamate «kanban») per rappresentare le attività, i materiali o i prodotti in un processo produttivo.

L

Lean

Lean è una metodologia di gestione della produzione e dei processi aziendali che mira a massimizzare il valore per il cliente riducendo gli sprechi e migliorando l'efficienza.

Originariamente sviluppata nel settore manifatturiero, la filosofia Lean si basa su principi come il miglioramento continuo, il rispetto per le persone e l'ottimizzazione del flusso di lavoro.

M

MQTT

Message Queuing Telemetry Transport, è un protocollo di messaggistica basata su un architettura publish/subscribe, progettato per essere leggero ed efficiente, particolarmente adatto per dispositivi con risorse limitate e reti con larghezza di banda ridotta.

N

O

P

PEM

Privacy-Enhanced Mail, è un formato di file utilizzato per memorizzare e trasmettere certificati digitali o chiavi private crittografati.

È codificato in Base64 e in questo caso viene usato per trasmettere i certificati X.509 di AWS IoT.

PFX

Personal Information Exchange, noto anche come PKCS12, è un formato di file utile per incorporare più certificati e chiavi private in un unico file, che può essere anche protetto da una password.

Q

Quality of Service

Per il protocollo MQTT il Quality of Service definisce con che garanzia verranno consegnati i messaggi ai dispositivi «destinatari» per un certo topic. Esistono tre livelli di Quality of Service: 0 (al massimo una consegna), 1 (almeno una consegna) e 2 (esattamente una consegna).

R

RFID

Radio-Frequency Identification, è una tecnologia di identificazione che utilizza onde radio per trasmettere dati tra un lettore e un dispositivo chiamato «tag» o «etichetta» RFID.

RSSI

Received Signal Strength Indicator, è una misura della potenza del segnale ricevuto da un dispositivo, come un lettore RFID, che indica la forza del segnale radio tra il lettore e il tag RFID.

S

T

Topic

In MQTT, un topic è un canale di comunicazione dove i dispositivi MQTT possono pubblicare messaggi su un specifico topic; tutti i dispositivi iscritti ad un topic riceveranno i messaggi pubblicati in esso.

Ogni topic consiste in una stringa che può essere suddivisa in livelli utilizzando il carattere «/» come separatore, consentendo di organizzare i messaggi in una struttura gerarchica.

U

UML

Unified Modeling Language, è un linguaggio di modellazione standardizzato utilizzato per visualizzare e documentare i componenti di un sistema software.

V

W

Wildcard

In MQTT, le wildcard sono caratteri speciali utilizzati nei topic per fare in modo che un client possa ricevere messaggi da più topic contemporaneamente, definendo un pattern gerarchico, dove uno o più livelli vengono resi variabili tramite i caratteri speciali «#» per rappresentare più livelli e «+» per rappresentare un singolo livello; per variabili si intende che ammettono qualsiasi stringa al loro posto.

X

Y

Z